A Project Report on

MediGuard Bot: ESP32-Powered Assistance Robot with Surveillance for Medicine Dispensing and Threat Detection

Submitted in partial fulfilment of the requirement for the award of the degree of

BACHELOR OF TECHNOLOGY
In
COMPUTER SCIENCE AND ENGINEERING
(Internet of Things)

Submitted by

K. Venu Oohitha	U. Sita Maha Lakshmi	M. Mahendra Babu	V. Chaitanya Krishna
21ME1A4929	21ME1A4962	21ME1A4932	22ME5A4905

Under the Esteemed Guidance of

Dr. P. Sudhakar
Associate Professor & HoD
Department of Computer Science and Engineering
(Internet of Things)

RAMACHANDRA COLLEGE OF ENGINEERING
(AUTONOMOUS)
(Approved by AICTE, New Delhi, Permanently Affiliated to JNTUK: Kakinada)
Accredited by NAAC A+ & NBA, An ISO 9001:2015 certified institution
NH16 Bypass Road, Vatluru (V), ELURU-534007, ELURU Dist., A.P,
Website: www.rcee.ac.in.

2024-2025

RAMACHANDRA COLLEGE OF ENGINEERING (AUTONOMOUS)

(Approved by AICTE, New Delhi, Permanently Affiliated to JNTUK: Kakinada)

Accredited by NAAC A+ & NBA, An ISO 9001:2015 certified institution

NH16 Bypass Road, Vatluru (V), ELURU-534007, ELURU. Dist., A.P,
Website: www.rcee.ac.in.

Department of Computer Science and Engineering (Internet of Things)

CERTIFICATE

This is to certify that **K. Venu Oohitha (21ME1A4929), U. Sita Maha Lakshmi (21ME1A4962), M. Mahendra Babu (21ME1A4932), V. Chaitanya Krishna (22ME5A4905)** students of **Bachelor of Technology** in **Computer Science and Engineering (Internet of Things)** have successfully completed their project work entitled **"MEDIGUARD BOT: ESP32-POWERED ASSISTANCE ROBOT WITH SURVEILLANCE FOR MEDICINE DISPENSING AND THREAT DETECTION"** at **Ramachandra College of Engineering, Eluru** during the **Academic Year 2024-2025**. This document is submitted in partial fulfilment for the award of the Degree of Bachelor of Technology in Computer Science and Engineering (IoT) and the same is not submitted elsewhere.

Dr. P. Sudhakar
Project Mentor

Dr. P. Sudhakar
HoD-CSE (IoT)

External Examiner

DECLARATION

We are, **K. Venu Oohitha (21ME1A4929), U. Sita Maha Lakshmi (21ME1A4962), M. Mahendra Babu (21ME1A4932), V. Chaitanya Krishna (22ME5A4905)** hereby declare the project report titled **"MEDIGUARD BOT: ESP32-POWERED ASSISTANCE ROBOT WITH SURVEILLANCE FOR MEDICINE DISPENSING AND THREAT DETECTION"** under the supervision of **Dr. P. Sudhakar,** Associate Professor & HoD, Department of Computer Science and Engineering (IoT) is submitted in partial fulfilment of the requirements for the award of the degree of Bachelor of Technology in Computer Science and Engineering (IoT).

This is a record of work carried out by us and the results embodied in this project have not been reproduced or copied from any source. The results embodied in this project report have not been submitted to any other University or Institute for the award of any other degree or diploma.

K. Venu Oohitha	U. Sita Maha Lakshmi	M. Mahendra Babu	V. Chaitanya Krishna
21ME1A4929	21ME1A4962	21ME1A4932	22ME5A4905

ACKNOWLEDGEMENT

We wish to take this opportunity to express our deep gratitude to all the people who have extended their cooperation in various ways during our project work. It is our pleasure and responsibility to acknowledge the help of all those individuals.

We sincerely thank our guide **Dr. P. Sudhakar, Associate Professor,** Department of Computer Science and Engineering (IoT) for helping us in successful completion of our project under his supervision**.**

We are very grateful to **Dr. P. Sudhakar, Head of the Department, Computer Science and Engineering (IoT)** for his assistance and encouragement in all respects in carrying throughout our project work.

We express our deepest gratitude to **Dr. M. Muralidhara Rao, Principal,** Ramachandra College of Engineering, Eluru for his valuable suggestions for successful completion of the project.

We express our deepest gratitude to the **Management** of Ramachandra College of Engineering, Eluru for their support and encouragement in completing our project work and providing us necessary facilities.

We sincerely thank all the faculty members of the Department of Computer Science and Engineering (IoT) for their valuable advices, suggestions and constant encouragement which played a vital role in carrying out this project work.

Finally, we thank one and all who directly or indirectly helped us to complete our project work successfully.

K. Venu Oohitha	**U. Sita Maha Lakshmi**	**M. Mahendra Babu**	**V. Chaitanya Krishna**
21ME1A4929	**21ME1A4962**	**21ME1A4932**	**22ME5A4905**

ABSTRACT

In modern healthcare and home security, timely medicine dispensing, real-time monitoring, and threat detection are crucial, especially for elderly and chronically ill patients. The MediGuard Bot is an ESP32-powered healthcare and security assistance robot that integrates automated medicine dispensing, environmental hazard detection, and real-time surveillance in a compact system. It utilizes an RTC-controlled CD drive motor for precise and scheduled medication delivery, while gas and flame sensors detect potential hazards and trigger instant alerts. An IP Camera enables live video streaming, providing remote surveillance via an IoT-based web interface. Unlike autonomous robots, the MediGuard Bot moves only when manually controlled, ensuring precise navigation for medicine delivery. In emergencies, it activates real-time alerts through a buzzer and LCD display. The ESP32 microcontroller efficiently manages sensor data and executes automated tasks. With Wi-Fi-based IoT connectivity, users can control and monitor the system remotely. Designed for hospitals, elderly care homes, and smart homes, the MediGuard Bot provides an intelligent, secure, and efficient solution for healthcare automation and home security.

CONTENTS

LIST OF FIGURES

LIST OF TABLES

CHAPTER 1
INTRODUCTION

1.1 INTRODUCTION

In modern healthcare and home security, the demand for automation has increased significantly to improve patient care, medicine management, and environmental safety. Many patients, particularly the elderly and those with chronic illnesses, require timely medication, but manual monitoring can result in missed doses or incorrect administration. Additionally, emergency threats such as gas leaks and fire hazards pose risks to patient safety, necessitating real-time detection and alert systems. The integration of IoT, robotics, and automation has revolutionized healthcare by providing smart solutions for patient assistance and security monitoring.

To address these challenges, the MediGuard Bot is designed as an IoT-powered, remote-controlled robotic system that integrates automated medicine dispensing, threat detection, and real-time surveillance in a single platform. The system is built using an ESP32 microcontroller, with an RTC module for scheduled medicine dispensing, an IP camera for real-time video monitoring, and gas and flame sensors to detect potential environmental hazards. Unlike fully autonomous robots, the MediGuard Bot moves only when manually controlled via a web-based interface, allowing users to remotely navigate it to deliver medicine or monitor a specific area. By combining automation, IoT connectivity, and smart surveillance, this project provides an efficient healthcare and home security solution, ensuring improved patient assistance and environmental safety.

1.2 MOTIVATION AND NEED

Many patients, particularly the elderly and those with chronic illnesses, struggle to adhere to their medication schedules, leading to serious health complications. Caregivers and healthcare facilities also face challenges in continuously monitoring patients and responding to emergencies such as gas leaks and fires. Traditional medicine dispensers lack real-time surveillance, remote accessibility, and automated threat detection, making it difficult for caregivers to provide timely assistance. Additionally, existing healthcare solutions are often

fragmented, focusing either on medicine dispensing or security, but not integrating both functionalities into a single, efficient system.

The MediGuard Bot is designed to bridge this gap by combining scheduled medicine dispensing, remote-controlled movement, real-time surveillance, and environmental hazard detection. Unlike standalone solutions, it offers a unified, IoT-enabled approach that allows caregivers to remotely monitor and control the system while ensuring patient safety. The integration of smart healthcare automation, IoT connectivity, and AI-driven monitoring makes this system a much-needed advancement for hospitals, elderly care facilities, and smart homes, addressing the growing demand for intelligent healthcare assistance and home security solutions.

1.3 PROBLEM STATEMENT

Patients, especially the elderly and those with chronic illnesses, require timely medication, continuous monitoring, and a secure environment to prevent health risks and emergencies. However, manual intervention in medicine management can lead to missed doses, improper administration, and delayed response to health threats. Additionally, home security concerns such as gas leaks, fires, and unauthorized access demand instant detection and real-time alerts to ensure safety. Existing solutions either focus solely on medicine reminders or security surveillance but lack an integrated, IoT-enabled robotic system that offers automated medicine dispensing, live surveillance, and environmental hazard detection in one platform. The MediGuard Bot is designed to bridge this gap by integrating remote-controlled movement, real-time healthcare monitoring, and smart automation, reducing human error and improving patient safety, healthcare efficiency, and home security in hospitals, elderly care facilities, and smart homes.

1.4 AIM & OBJECTIVES OF STUDY

The primary aim of this project is to design and develop an IoT-enabled healthcare assistance robotic system that integrates automated medicine dispensing, threat detection, real-time surveillance into a single platform to enhance patient safety, home security, and healthcare automation through remote monitoring, scheduled medication delivery, and environmental hazard detection.

Objectives:

- Automate medicine dispensing using an RTC module for precise scheduling.

- Detect potential hazards such as gas leaks and fire through sensor integration.
- Enable real-time video monitoring using an IP-Cam.
- Provide IoT-based remote monitoring and control via a web interface.
- Generate alerts using a buzzer, voice module, and LCD display for effective notifications.
- Ensuring seamless IoT connectivity for real-time communication and remote access.
- Providing a battery-powered, mobile, and compact design suitable for healthcare and home security applications.

1.5 ORGANIZATION OF THE PROJECT

This project is structured to cover all essential aspects of the MediGuard Bot, from conceptualization to implementation. The initial sections introduce the problem statement, research motivation, and objectives, highlighting the need for an automated medicine dispensing and patient safety system. The literature review explores existing healthcare automation solutions, analyzing their limitations and identifying how the proposed system improves upon them. The methodology section details the hardware and software components, including the ESP32 microcontroller, CD drives for dispensing, IoT modules, and sensor integrations for hazard detection. The system architecture and working principles are explained with block diagrams, circuit connections, and software flowcharts.

Subsequent sections focus on development, implementation, and testing of the MediGuard Bot. The hardware integration process is explained, covering motor mechanisms for medicine dispensing, robot movement, and real-time monitoring using the IP Camera. The software development part includes Arduino programming for hardware control, React.js for the web interface, and Node.js for backend operations. The testing and evaluation section presents the performance analysis, accuracy of dispensing, and responsiveness of the IoT system. Finally, the project concludes with results, future enhancements, and potential applications in healthcare environments.

CHAPTER 2
LITERATURE REVIEW

The integration of automation and IoT in healthcare robotics has seen a significant rise, particularly in supporting medicine dispensing and real-time monitoring. A study by Ramesh et al. developed a robot to deliver medicines and monitor patient vitals using an ESP32 module, aligning closely with our project's medicine dispensing functionality but lacking our web-based remote surveillance interface [1]. Similarly, a model proposed by Dubey et al. featured an ESP32 CAM-based surveillance robot, supporting threat detection via gas and flame sensors, which resonates with the environmental hazard detection aspects of our bot [2]. The use of IoT for automatic medicine dispensing with real-time scheduling was also explored in an ESP32-based model, though it did not incorporate manual-controlled navigation or IP surveillance like the MediGuard Bot [3].

Advanced hospital robots combining basic medicine delivery and patient care features were described by Asharuf et al., though their reliance on manual interaction differs from our system's integrated real-time alerting and remote monitoring [4]. Another system by Ismail et al. emphasized pill dispensing and vitals tracking using microcontrollers but lacked visual surveillance, making our solution more secure [5]. Incorporation of RTC-based scheduling in a healthcare robot, as proposed in [6], aligns with our RTC-controlled motorized medicine delivery system. Alexa-integrated solutions were seen in smart medicine reminders using ESP and AWS IoT, though they did not address threat detection or manual navigation features [7].

Several models focus primarily on surveillance, like the ESP32 CAM robot developed in [8], but fail to offer medicine dispensing capabilities, making MediGuard more multifunctional. A hybrid solution involving automatic pill dispensing and obstacle avoidance was presented in [9], which though intelligent, does not provide user-controlled mobility or real-time hazard alerts. A detailed framework for remote medicine distribution and automation using IoT sensors is illustrated in [10], supporting our approach with enhanced connectivity for real-time monitoring. Another project on scheduled medicine reminders with display and alert systems validates our system's buzzer and LCD alert setup [11].

The importance of intelligent healthcare assistance using robotic platforms is addressed in a study that developed a mobile robot for elderly care with minimal

threat detection functionalities [12]. In contrast, our project uniquely combines IP camera surveillance and multi-sensor threat detection for enhanced safety. Real-time alert systems in IoT environments for gas leak detection, as developed in [13], mirror our flame and gas sensor-based emergency module. Likewise, similar threat monitoring applications with buzzer and LCD alerts are implemented in [14], validating our system's emergency response design.

Research on AI-assisted elderly monitoring systems through social robotics and ambient intelligence [15] highlights the role of assistive automation but lacks specific integration with medicine dispensing features. An improved IoT-based health assistant capable of scheduled alerts and cloud integration [16] confirms the significance of remote control and real-time feedback, both embedded in our system. Enhanced surveillance robots with Wi-Fi-based streaming capabilities, though equipped with AI, lacked RTC-based dispensing features [17].

A low-cost medicine delivery robot using GPS and sensors demonstrated effective location tracking but did not offer manual-controlled precision like our MediGuard Bot [18]. Another intelligent alerting system focused on gas leakage detection in industrial zones validated our sensor approach, albeit in a different domain [19]. Early fire detection combined with smart alerts in IoT sensor networks as discussed in [20] complements the alerting module of our bot. Other works like [21] proposed intelligent robotics for home security and alerting, focusing on motion detection, which supports our choice of threat detection integration.

IP camera-based real-time streaming solutions, like those implemented in [22], affirm the decision to exclude ESP32 CAM in favour of more stable surveillance methods. Innovations in health automation and medicine delivery, like those in [23], support the use of microcontroller-based modules for precise dispensing systems. Threat detection through improved smart sensor designs in home environments, as developed in [24], align well with the gas and flame detection mechanisms of MediGuard. Finally, hybrid alert systems for LPG and fire prediction through smart IoT architectures were explored in [25], validating the utility of our buzzer, sensors, and autonomous monitoring system.

From this survey, it is evident that while significant advancements have been made in medicine dispensing, threat detection, and surveillance, there remains a gap in integrating these functionalities into a single, IoT-enabled robotic system. The MediGuard Bot is designed to address these challenges for smart healthcare.

CHAPTER 3

EXISTING SYSTEM

3.1 OVERVIEW OF TRADITIONAL SYSTEMS

The field of healthcare robotics has seen significant advancements in recent years, particularly in the development of medicine dispensing and patient monitoring systems. Automated medicine dispensers are widely used in healthcare facilities and home care settings to ensure medication adherence. These systems often integrate timers and sensors to dispense medications at scheduled intervals. Many of these dispensers are designed to alert caregivers or patients via notifications when medication needs to be taken. However, these dispensers typically lack additional functionalities, such as mobility or surveillance, which limits their usability in comprehensive healthcare scenarios.

Robotic assistance in elderly care is another key area where existing systems have gained prominence. Several robots are designed to assist elderly or disabled individuals by providing medication reminders, health tracking, and companionship. These systems often use voice assistance to deliver reminders and provide simple health monitoring capabilities, such as heart rate tracking. While these systems address some healthcare challenges, they are stationary and unable to physically deliver medication to patients who require assistance in different locations, thereby restricting their functionality.

IoT-enabled healthcare systems have further enhanced patient care by enabling remote monitoring and control. These systems utilize internet-connected devices to collect and transmit patient data to caregivers or healthcare providers in real time. In some cases, IoT-based medicine dispensers allow caregivers to remotely control the release of medications. However, these systems are often limited to stationary use and do not integrate real-time video surveillance or hazard detection, which are critical for patient safety in home healthcare.

Surveillance robots with mobility have also been developed for home security and healthcare applications. These robots integrate camera modules, such as the ESP32-CAM, to provide live video feeds to users for remote monitoring. They are often equipped with sensors to detect environmental hazards, such as gas leaks or fire, and send real-time alerts. Despite their mobility and surveillance capabilities,

these robots lack functionalities related to medicine dispensing and patient health monitoring, making them unsuitable for healthcare-specific applications.

Autonomous hospital logistics robots are another category of systems that have gained attention in medical facilities. These robots are designed to transport medical supplies, including medications, to specific locations within hospitals. They use advanced navigation systems to autonomously move through hospital corridors and deliver items efficiently. However, these systems are fully autonomous and do not offer manual control options, making them less adaptable to smaller healthcare settings or home environments. Moreover, they focus primarily on logistics and do not include features such as real-time surveillance or patient monitoring.

3.2 LIMITATIONS OF TRADITIONAL SYSTEMS

1. **Limited Integration of Functionalities**: Most existing systems focus on either medicine dispensing, patient monitoring, or surveillance, without offering a comprehensive solution that integrates all three functionalities.
2. **Lack of Mobility in Medicine Dispensers**: Automated medicine dispensers are stationary and cannot physically deliver medications to patients in different locations, reducing their usability in dynamic environments.
3. **No Real-Time Surveillance**: Many healthcare systems lack integrated video surveillance, making it difficult for caregivers to monitor patients or detect environmental hazards in real time.
4. **Absence of Environmental Hazard Detection**: Existing systems often do not include sensors to detect fire, gas leaks, or other environmental hazards, which are critical for ensuring patient safety.
5. **Stationary IoT Systems**: IoT-enabled systems for healthcare are mostly stationary and do not support mobility, making them unsuitable for tasks that require movement or physical interaction with patients.
6. **Autonomy Without Manual Control**: Autonomous hospital robots lack manual control options, which limits their adaptability for personalized or small-scale healthcare settings.
7. **Inadequate Support for Elderly Patients**: Robotic assistance systems for elderly care often rely solely on voice reminders and notifications, without offering physical medicine dispensing or delivery capabilities.

8. **High Implementation Costs**: Many existing systems, particularly autonomous robots, involve high implementation and maintenance costs, making them inaccessible for home healthcare users.
9. **Limited Adaptability in Home Settings**: Existing robots designed for hospitals are often too complex or large to be used effectively in home environments, where space and user interaction requirements differ.
10. **No Multi-Functional Design**: Most existing systems are designed for single-purpose applications, such as medicine dispensing or surveillance, and lack the multi-functional capabilities needed for comprehensive healthcare solutions.

CHAPTER 4

PROPOSED SYSTEM

4.1 OVERVIEW OF PROPOSED SYSTEM

The MediGuard Bot is a manually controlled IoT-enabled healthcare robot designed to assist in medicine dispensing, real-time video surveillance, and environmental hazard detection. Unlike autonomous systems, the MediGuard Bot relies entirely on manual navigation through a web-based interface, accessible via mobile devices or computers. This interface serves as the primary control hub, allowing users to issue movement commands, monitor live video feeds, and manage medication schedules remotely, ensuring precise control and adaptability in various healthcare environments.

The web-based interface offers a user-friendly platform for controlling the bot and managing healthcare tasks. Through this interface, users can remotely schedule and monitor medication dispensing, receive real-time notifications, and access alerts related to environmental hazards, such as gas leaks or fires. The interface is specifically designed to simplify healthcare management for caregivers and healthcare providers, enabling better patient safety, improved medication adherence, and reduced caregiver workload. By centralizing control and monitoring, the system provides enhanced support for elderly or dependent individuals who require consistent care.

The bot's medicine dispensing mechanism is a standout feature, integrating a CD drive motor and a real-time clock (RTC) module for precise and timely pill release. Caregivers can use the web interface to configure schedules, ensuring medication is dispensed at the right time without manual intervention. This feature significantly reduces the chances of missed doses, improving medication adherence for patients with complex prescription regimens. Additionally, real-time notifications alert caregivers and patients about dispensing schedules, offering an added layer of reliability and convenience.

The MediGuard Bot enhances safety through its environmental hazard detection capabilities. Equipped with sensors, the bot continuously monitors for risks such as gas leaks, fires, and unauthorized motion. On detecting a hazard, the system instantly notifies the user through the web interface, enabling swift and appropriate

action. This feature, combined with live video surveillance via the IP camera, ensures comprehensive home and healthcare security, allowing users to maintain a watchful eye on patients and their environment from any location.

Manual movement of the bot ensures precise navigation in complex or crowded environments under the direct control of the user. This eliminates the challenges of autonomous navigation, making the system cost-effective, reliable, and user-friendly. The integration of manual movement with advanced functionalities like medicine dispensing, environmental monitoring, and real-time surveillance ensures that the MediGuard Bot addresses modern healthcare challenges effectively. By combining user-friendly remote control, automated healthcare features, and enhanced security, the system provides a holistic solution for patient care and caregiver support in both home and institutional settings.

4.2 BLOCK DIAGRAM

The block diagram represents the core architecture of the MediGuard Bot system, an ESP32-powered robot designed for healthcare assistance, including medicine dispensing, environmental monitoring, and real-time surveillance.

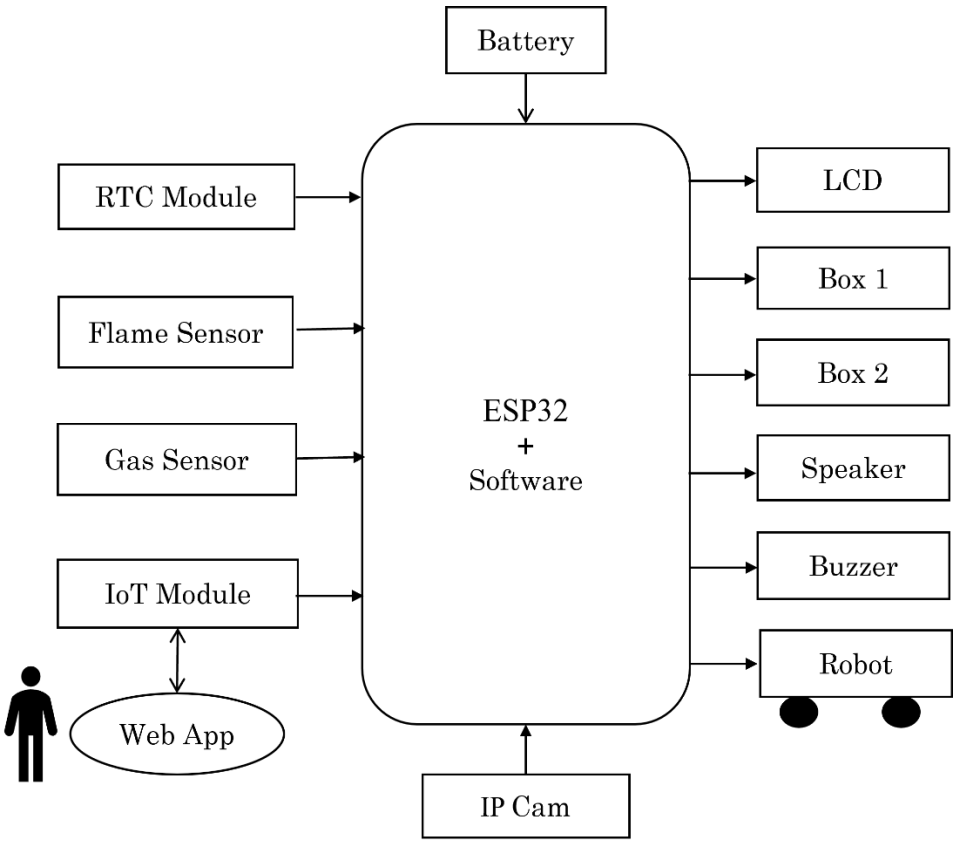

Fig. 4.1: Block Diagram of MediGuard Bot

At its core, the ESP32 microcontroller integrates all components, coordinating their functionalities through software. On the input side, the system incorporates several modules for diverse functionalities. The RTC Module ensures accurate scheduling of medication dispensing. Environmental safety is managed by the Flame Sensor and Gas Sensor, which detect fire and hazardous gases. The IoT Module facilitates remote communication, allowing data exchange between the robot and the user through a web-based interface. The Web App, a critical component, serves as the user interface for controlling the bot's movement, scheduling medications, and receiving real-time notifications. Additionally, the IP Camera provides live video streaming for surveillance, ensuring effective monitoring of patients and their surroundings.

On the output side, the system includes various modules for delivering its services. The LCD displays key information such as system alerts and medication schedules. The Box 1 and Box 2 units store and dispense different medications as per the schedule. The Voice Module offers auditory feedback for alerts and confirmations, while the Buzzer provides additional sound signals during emergencies. The Robot base is manually navigated via the web interface, allowing users to move the bot to different locations.

The system is powered by a battery designed to suit the MediGuard Bot's mobility and operational requirements, ensuring uninterrupted and portable functionality. The ESP32 acts as the central hub, processing commands from the web app, collecting sensor data, and managing the dispensing and monitoring tasks. This integration enables the MediGuard Bot to serve as a versatile tool for enhancing patient care, especially for elderly or dependent individuals, by providing remote monitoring, medication adherence, and hazard detection.

4.3 WORKING

The MediGuard Bot is a manual control-based robot designed to enhance healthcare assistance by integrating medicine dispensing, hazard detection, real-time surveillance, and user interaction through a web-based interface. The system operates in a structured and user-friendly manner, ensuring seamless functionality across its various modules.

1. **System Initialization:**

   The operation begins with the initialization of the ESP32 microcontroller, which acts as the main control unit. All connected components, such as the RTC module, sensors, and IoT module, are activated and synchronized for proper functioning.

2. **Medicine Scheduling and Dispensing:**

   The RTC (DS3231) module ensures precise timekeeping to schedule medication dispensing. The system checks the current time against the present schedule. If it matches, the corresponding medicine box is activated using CD drivers to dispense the required medication. Notifications are sent to the user via the web-based interface to confirm the action.

3. **Real-Time Surveillance:**

   The IP-CAM module captures live video feeds, which are streamed to the web-based interface. This feature allows users or caregivers to monitor the environment or the patient in real time, ensuring security and patient safety.

4. **Hazard Detection and Alerts:**

   The flame sensor (KY-026) and gas sensor (MQ-2) continuously monitor the environment for fire and gas leaks, respectively. If any hazard is detected, the system immediately triggers the buzzer to emit an audible alarm and sends a real-time alert to the web application. These features ensure timely hazard response.

5. **Manual Robot Navigation:**

   The mobility of the MediGuard Bot is controlled through manual commands sent from the web-based interface, accessible on a mobile device or computer. The user can navigate the robot in forward, backward, left, and right directions. The robot's motors and chassis ensure smooth movement, allowing it to deliver medications or perform surveillance in different locations.

6. **User Interaction via Web Application:**

   The web-based interface serves as the central control and monitoring platform. It allows users to input medicine schedules, control robot movement, monitor live video feeds, and receive notifications or alerts. The IoT module (ESP8266) ensures reliable internet connectivity for remote operation.

7. **Status Display and Audio Feedback:**

   The 16x2 LCD displays system status, such as ongoing operations, connection status, and alerts. Simultaneously, the voice module (APR33A3) delivers pre-recorded audio notifications for enhanced user accessibility.

8. **Power Supply and Circuit Assembly:**

   A rechargeable 12V Li-ion battery powers the entire system, providing portability and uninterrupted operation. The connections between components are established using jumper wires and soldered onto a breadboard for stability and reliability.

4.4 CIRCUIT DIAGRAM

The circuit diagram of the MediGuard Bot demonstrates an integrated system centred around the ESP32 microcontroller. The power supply originates from a battery connected through a switch to a bridge rectifier consisting of four diodes (D1–D4), which converts AC to DC. This is followed by a 7805-voltage regulator that provides a stable 5V output, further filtered using 1000µF and 100µF capacitors, with an LED indicator to show the power status. The ESP32 acts as the brain of the system, interfacing with various peripherals via its GPIO pins. Two motor drivers labelled Drive 1 and Drive 2 are responsible for the medicine dispensing operation, controlled by the ESP32 through designated GPIOs.

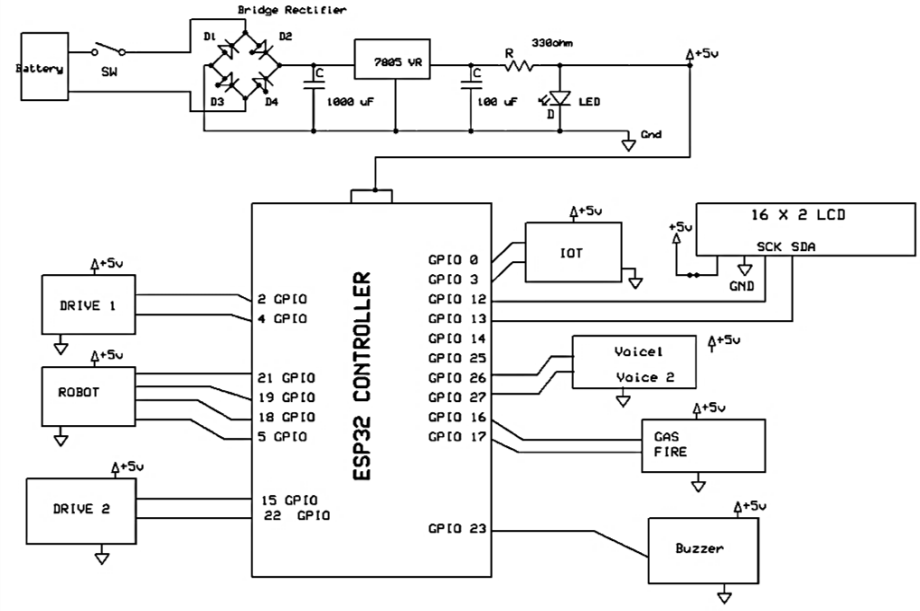

Fig. 4.2: Circuit Diagram of MediGuard Bot

A separate motor-controlled module labelled ‘robot’ handles the movement of the robot. An IoT module is included in the circuit to enable remote access and monitoring capabilities, ensuring real-time data transmission. The system also features a 16x2 LCD interfaced via I2C protocol to display real-time system status and sensor readings. Voice module is connected to provide audio instructions regarding medicine dispensing, enhancing user interaction. For threat detection, gas and fire sensors are integrated, feeding input to the ESP32 for immediate analysis. In case of danger, a buzzer is triggered by the controller to issue an audible alarm, completing the security and surveillance function of the bot.

4.5 FLOW CHART

The flowchart outlines the operational workflow of the MediGuard Bot, demonstrating its step-by-step functionality. It begins with the initialization of the ESP32 microcontroller and associated sensors, including flame and gas sensors, to prepare the system for monitoring and task execution. The system checks the RTC (Real-Time Clock) module for any scheduled medication times. If a schedule is detected, the bot dispenses the medication from its compartments and sends a notification to the user, ensuring timely adherence. If no schedule is identified, the system continues to monitor other activities.

The bot continuously monitors environmental conditions using its flame and gas sensors. Upon detecting a hazard, such as fire or gas leakage, the bot triggers an alert through a buzzer and sends notifications for immediate response. In the absence of hazards, it proceeds with its regular monitoring and surveillance functions. The bot’s real-time surveillance capabilities, enabled by the IP-CAM module, allow live video streaming to users through a web-based interface. Users can interact with the bot via this platform to issue commands for manual navigation or access system features.

When navigation is activated, the bot processes user commands to move forward, backward, left, or right, allowing it to reach desired locations. If no movement commands are issued, the bot remains stationary, focusing on its monitoring tasks. The workflow also includes a stop command option, enabling users to halt all operations instantly. Upon stopping, the bot returns to standby mode while continuing its monitoring functions.

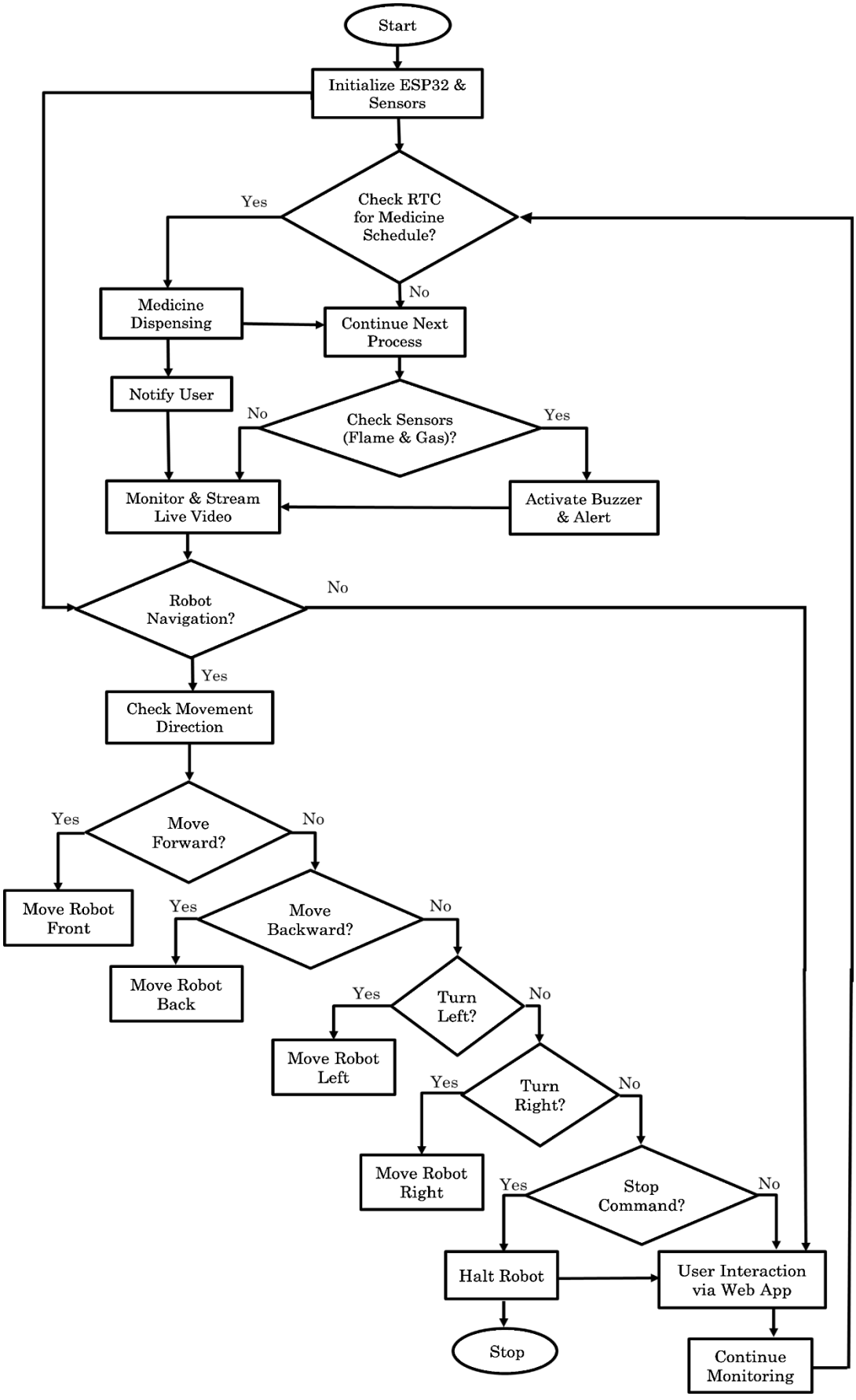

Fig. 4.3: Flow Diagram of MediGuard Bot

This workflow ensures the MediGuard Bot operates efficiently, providing a structured and user-controlled solution for medication dispensing, hazard detection, real-time surveillance, and patient safety.

4.6 ADVANTAGES

1. **Comprehensive Healthcare Assistance:**
   Combines medicine dispensing, hazard detection, and surveillance into one system, ensuring timely medication and patient safety.

2. **Real-Time Monitoring and Alerts:**
   Enables live video monitoring and hazard detection with instant alerts for emergencies, ensuring quick responses.

3. **Remote and Manual Control:**
   Allows remote navigation and control via a web-based interface, enabling precise robot operation for caregivers.

4. **Improved Medication Adherence:**
   Ensures timely medicine dispensing using an RTC module, reducing human errors in medication management.

5. **Enhanced Safety Measures:**
   Detects hazards like fire and gas leaks, triggering buzzer alerts and notifications to prevent accidents.

6. **User-Friendly Interaction:**
   Provides easy scheduling, navigation, and notifications via a web app, LCD display, and audio feedback for accessibility.

7. **Portability and Versatility:**
   Mobile chassis and rechargeable battery allow portability, making it suitable for home and clinical environments.

8. **Reduced Caregiver Workload:**
   Automates routine tasks like medicine dispensing and monitoring, freeing caregivers for other essential duties.

9. **Cost-Effective and Scalable:**
   Uses affordable components and offers a modular design for future upgrades and scalability.

10. **Promotes Independence for Patients:**
    Empowers users to manage medications and safety independently with minimal caregiver assistance.

4.7 APPLICATIONS

1. **Home Healthcare:**
   Ideal for elderly or disabled individuals, the bot ensures timely medication delivery, real-time monitoring, and enhanced safety in home environments.

2. **Hospitals and Clinics:**
   Supports medical staff by automating medicine dispensing, monitoring patients, and detecting hazards in healthcare facilities.

3. **Elderly Care Centers:**
   Assists caregivers by managing medicine schedules, monitoring residents, and providing real-time alerts for emergencies.

4. **Rehabilitation Centers:**
   Facilitates recovery by delivering medicines and monitoring patients in rehabilitation facilities.

5. **Remote Patient Monitoring:**
   Allows caregivers to monitor patients in distant locations through real-time video and alerts, ensuring proper care and safety.

6. **Disaster Relief Shelters:**
   Useful in emergency shelters for dispensing medicines and detecting hazards like gas leaks or fire.

7. **Standalone Surveillance:**
   Functions as a mobile surveillance system to monitor and secure sensitive areas in healthcare or other environments.

8. **Smart Home Integration:**
   Can be integrated into smart home systems for enhanced health monitoring and safety automation.

9. **Educational Purposes:**
   Acts as a project model for robotics and healthcare technology research and development.

10. **IoT-Based Healthcare Solutions:**
    Demonstrates how IoT and robotics can collaborate to improve patient care and safety in diverse healthcare settings.

CHAPTER 5

IOT - EMBEDDED SYSTEMS

An IoT-embedded system is a specialized computing system integrated into IoT devices. It consists of hardware (microcontrollers, sensors, communication modules) and software (firmware, cloud integration) to collect, process, and transmit data for smart applications. IoT-embedded systems are revolutionizing industries by enabling real-time monitoring, automation, and intelligent decision-making.

5.1 INTRODUTION TO IOT - EMBEDDED SYSTEMS

An embedded system can be thought of as a computer hardware system having software embedded in it. An embedded system can be an independent system or it can be a part of a large system. An embedded system is a microcontroller or microprocessor-based system which is designed to perform a specific task. For example, a fire alarm is an embedded system; it will sense only smoke. A system consists of hardware, application software, and a Real-Time Operating System (RTOS). The hardware forms the physical foundation, enabling the execution of various tasks. Application software provides functionality and user interaction, allowing the system to perform specific operations. The RTOS plays a crucial role in managing system resources efficiently, ensuring real-time processing and responsiveness. Together, these components work seamlessly to deliver a reliable and efficient system.

Fig. 5.1: Overview of IoT-Embedded System

That supervises the application software and provides mechanism to let the processor run a process as per scheduling by following a plan to control the latencies. RTOS defines the way the system works. It sets the rules during the execution

application program. A small-scale embedded system may not have RTOS. So, we can define an embedded system as a Microcontroller based, software driven, reliable, real-time control system.

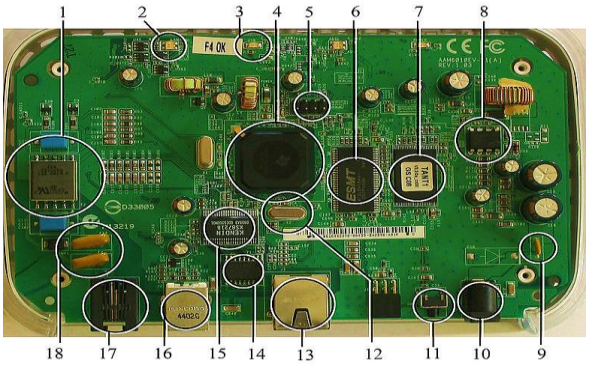

Fig. 5.2: A Modern Example of Embedded System

An embedded system is designed to perform a single specialized function repeatedly. For example, a pager continuously operates as a pager without deviation. These systems are tightly constrained, meaning they must adhere to strict design metrics, including cost, size, power consumption, and performance. They are often designed to fit on a single chip, operate efficiently in real time, and consume minimal power to extend battery life. Embedded systems are also reactive and real-time, requiring them to respond promptly to environmental changes. For instance, a car's cruise control system constantly monitors speed and brake sensors, making rapid calculations to adjust acceleration or deceleration. Any delay in processing could result in system failure. Additionally, embedded systems are microprocessor or microcontroller-based and require memory, usually embedded in ROM, eliminating the need for secondary storage. They are designed to be connected, allowing integration with peripherals for input and output functionality. Furthermore, embedded systems function as hardware-software (HW-SW) systems, where software enhances flexibility and features, while hardware ensures performance and security.

5.2 STRUCTURE OF EMBEDDED SYSTEM

An embedded system consists of several key components that work together to process and control data. A sensor measures a physical quantity, such as temperature or pressure, and converts it into an electrical signal that can be read by an observer or an electronic instrument like an Analog-to-Digital (A-D) converter.

The A-D converter then transforms the analog signal from the sensor into a digital format that can be processed by the system. The processor and Application-Specific Integrated Circuits (ASICs) play a crucial role in processing the data, measuring the output, and storing it in memory. Once the data is processed, a Digital-to-Analog (D-A) converter converts the digital data back into an analog format when needed. The actuator then compares the output from the D-A converter to the expected result and stores the approved output for execution. At the core of an embedded system is the processor, which serves as its heart by receiving inputs, processing data, and generating the appropriate output. For an embedded system designer, understanding both microprocessors and microcontrollers is essential, as they form the foundation of efficient and reliable system design. Processors in a System has two essential units- Program Flow Control Unit (CU), Execution Unit (EU).

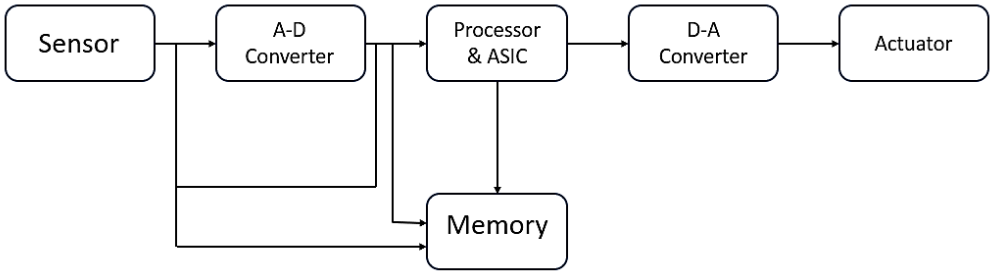

Fig. 5.3 :Structure of an Embedded System

The CU includes a fetch unit for fetching instructions from the memory. The EU has circuits that implement the instructions pertaining to data transfer operation and data conversion from one form to another. The EU includes the Arithmetic and Logical Unit (ALU) and also the circuits that execute instructions for a program control task such as interrupt, or jump to another set of instructions.

A processor runs the cycles of fetch and executes the instructions in the same sequences they are fetched from memory. Types of Processors Processors can be of the following categories such as General-Purpose Processor (GPP), Microprocessor, Microcontroller, Embedded Processor Digital Signal Processor, Media Processor, Application Specific System Processor (ASSP) and Application Specific Instruction Processors (ASIPs)

The Embedded system hardware includes elements like user interface, Input/Output interfaces, display and memory etc. Generally, an embedded system

comprises power supply, processor, memory, timers, serial communication ports and system application specific circuits.

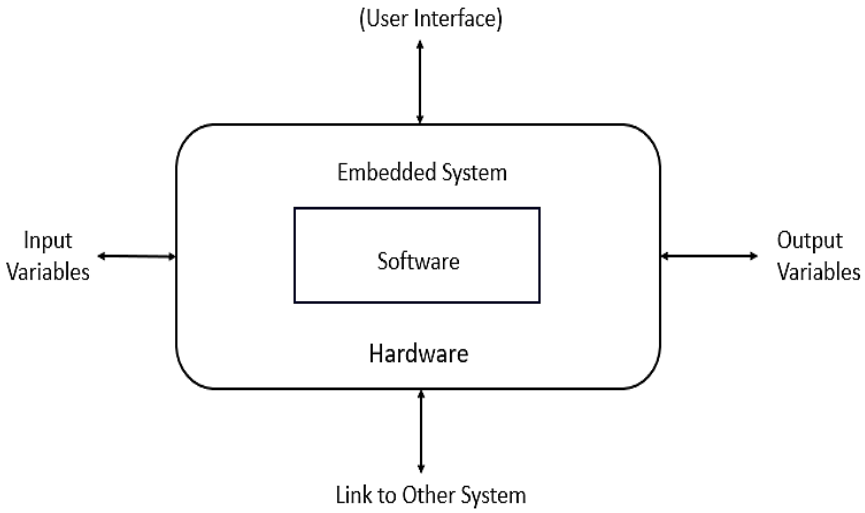

Fig. 5.4: Elements in Embedded System

5.3 SOFTWARE ARCHITECTURE

There are several different types of software architecture in common use.

Simple Control Loop: In this design, the software simply has a loop. The loop calls subroutines, each of which manages a part of the hardware or software.

Interrupt Controlled System: Some embedded systems are predominantly interrupt controlled. This means that tasks performed by the system are triggered by different kinds of events. An interrupt could be generated for example by a timer in a predefined frequency, or by a serial port controller receiving a byte. These kinds of systems are used if event handlers need low latency and the event handlers are short and simple. Usually, these kinds of systems run a simple task in a main loop also, but this task is not very sensitive to unexpected delays. Sometimes the interrupt handler will add longer tasks to a queue structure. Later, after the interrupt handler has finished, these tasks are executed by the main loop. This method brings the system close to a multitasking kernel with discrete processes.

Primitive Multitasking: In this type of system, a low-level piece of code switches between tasks or threads based on a timer (connected to an interrupt). This is the level at which the system is generally considered to have an "operating system" kernel. Depending on how much functionality is required, it introduces more or less of the complexities of managing multiple tasks running conceptually in parallel. As any code can potentially damage the data of another task (except in larger systems

using an MMU) programs must be carefully designed and tested, and access to shared data must be controlled by some synchronization strategy, such as message queues, semaphores or a non-blocking synchronization scheme.

Microkernels And Exokernels: A microkernel is a logical step up from a real-time OS. The usual arrangement is that the operating system kernel allocates memory and switches the CPU to different threads of execution. User mode processes implement major functions such as file systems, network interfaces, etc. In general, microkernels succeed when the task switching and intertrack communication is fast, and fail when they are slow. Exokernels communicate efficiently by normal subroutine calls. The hardware and all the software in the system are available to, and extensible by application programmers.

5.4 TYPES OF EMBEDDED SYSTEMS

Based on performance, functionality, requirement of the microcontroller, the embedded systems are divided into three categories:

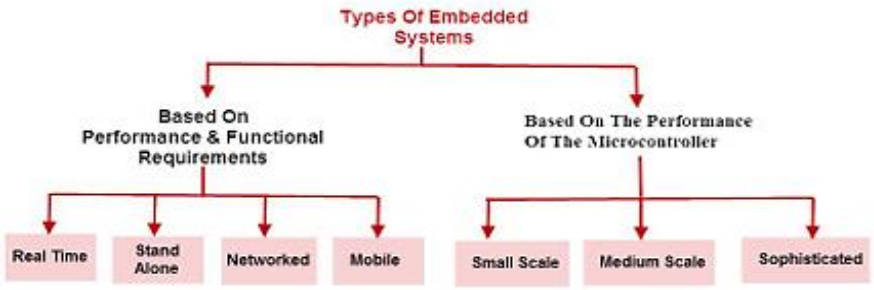

Figure 5.5: Classification of Embedded System

Types of Embedded systems embedded systems are classified into four categories based on their performance and functional requirements:

Stand Alone Embedded Systems: These systems do not require a host system like a computer, it works by itself. It takes the input from the input ports either analog or digital and processes, calculates and converts the data and gives the resulting data through the connected device-Which either controls, drives and displays the connected devices. Examples for the stand-alone embedded systems are mp3 players, digital cameras, video game consoles, microwave ovens and temperature measurement systems.

Real Time Embedded Systems: A system which gives a required o/p in a particular time. These types of embedded systems follow the time deadlines for

completion of a task. Real time embedded systems are classified into two types such as soft and hard real time systems.

Networked Embedded Systems: These types of embedded systems are related to a network to access the resources. The connected network can be LAN, WAN or the internet. The connection can be any wired or wireless. This type of embedded system is the fastest growing area in embedded system applications. The embedded web server is a type of system wherein all embedded devices are connected to a web server and accessed and controlled by a web browser. Example for the LAN networked embedded system is a home security system wherein all sensors are connected and run on the protocol TCP/IP.

Mobile Embedded Systems: Mobile embedded systems are used in portable embedded devices like cell phones, mobiles, digital cameras, mp3 players and personal digital assistants, etc. The basic limitation of these devices is the other resources and limitation of memory.

Embedded Systems are classified into three types based on the performance of the microcontroller such as- small scale embedded systems, medium scale embedded systems, and sophisticated embedded systems.

Small Scale Embedded Systems: These types of embedded systems are designed with a single 8 or 16-bit microcontroller, that may even be activated by a battery. For developing embedded software for small scale embedded systems, the main programming tools are an editor, assembler, cross assembler and integrated development environment (IDE).

Medium Scale Embedded Systems: These types of embedded systems design with a single or 16 or 32bit microcontroller, RISCs or DSPs. These types of embedded systems have both hardware and software complexities. For developing embedded software for medium scale embedded systems, the main programming tools are C, C++, JAVA, Visual C++, RTOS, debugger, source code engineering tool, simulator and IDE.

Sophisticated Embedded Systems: These types of embedded systems have enormous hardware and software complexities, that may need ASIPs, IPs, PLAs, scalable or configurable processors. They are used for cutting-edge applications that need hardware and software Co-design and components which have to assemble in the final system. Applications of Embedded Systems: Embedded systems are used in

different applications like automobiles, telecommunications, smart cards, missiles, satellites, computer networking and digital consumer electronics.

5.5 APPLICATIONS OF EMBEDDED SYSTEM

Embedded systems play a crucial role across various industries, enhancing efficiency, automation, and real-time processing. In the automotive sector, the power essential features such as Engine Control Units (ECUs), Anti-lock Braking Systems (ABS), airbags, Adaptive Cruise Control (ACC), and infotainment with GPS navigation. Consumer electronics heavily rely on embedded systems, found in smartphones, tablets, smart TVs, set-top boxes, digital cameras, gaming consoles, and wearable devices like smartwatches and fitness trackers. In medical and healthcare, embedded systems are integral to devices like pacemakers, infusion pumps, MRI and CT scanners, patient monitoring systems, and Automated External Defibrillators (AEDs). Industrial and automation systems also benefit from embedded technology, utilizing Programmable Logic Controllers (PLCs), robotics, SCADA systems, and smart sensors for efficient operations.

With the rise of the Internet of Things (IoT) and smart home automation, embedded systems are used in smart lighting, HVAC, security systems, connected appliances, and IoT-enabled wearables. In the aerospace and defense sector, they enable autopilot systems, missile guidance, satellite communications, and drone operations. Telecommunications rely on embedded systems in routers, modems, cell towers, and VoIP devices to ensure seamless connectivity. Smart agriculture and environmental monitoring leverage embedded technology for automated irrigation, crop monitoring via drones, and livestock health tracking. In smart energy and power systems, embedded systems facilitate smart meters for electricity, water, and gas, solar inverters, and energy harvesting solutions. Lastly, retail and payment systems incorporate embedded technology in Point of Sale (POS) terminals, automated vending machines, and RFID-based inventory management, streamlining transactions and logistics. These diverse applications highlight the versatility and significance of embedded systems in modern technology.

CHAPTER 6

HARDWARE DESCRIPTION

The hardware modules of an embedded system consist of several key components that work together to ensure efficient functionality. The processor, often a microcontroller or microprocessor, serves as the brain of the system, executing instructions and processing data. Memory is essential for storing program code and data, with ROM used for permanent storage and RAM for temporary operations. Input devices, such as sensors, gather data from the environment, while output devices, including displays, LEDs, or actuators, provide system responses. Power supply units ensure stable energy delivery, enabling uninterrupted operation. Communication interfaces, such as UART, SPI, I2C, or USB, facilitate data exchange between different components or external devices. Additionally, timers and counters help manage time-dependent tasks, ensuring precise execution.

6.1 ESP32 CONTROLLER

ESP32 is the SoC (System on Chip) microcontroller which has gained massive popularity recently. Whether the popularity of ESP32 grew because of the growth of IoT or whether IoT grew because of the introduction of ESP32 is debatable. If you know 10 people who have been part of the firmware development for any IoT device, chances are that 7−8 of them would have worked on ESP32 at some point.

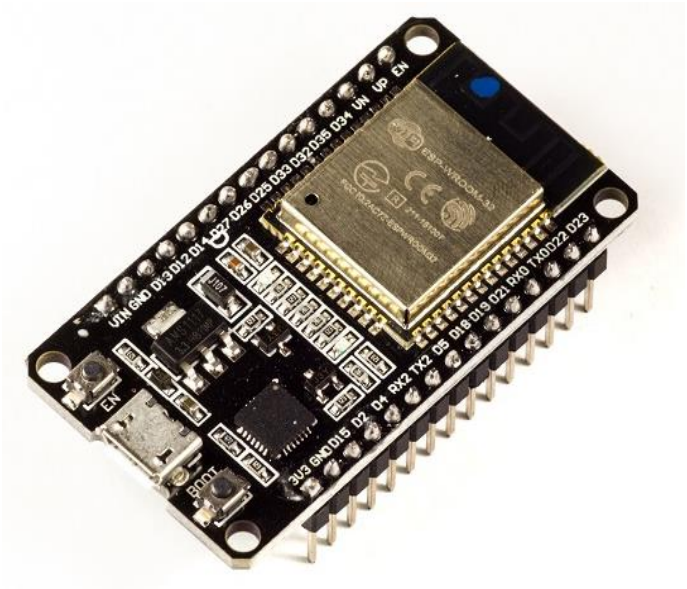

Fig. 6.1: ESP32 Microcontroller

6.1.1 Introduction to ESP32

The specs listed below belong to the ESP32 WROOM 32 variant such as Integrated Crystal (40 MHz), Module Interfaces (UART, SPI, I2C, PWM, ADC, DAC, GPIO, pulse counter, capacitive touch sensor), Integrated SPI flash (4 MB), ROM (448 KB for booting and core functions), SRAM (520 KB), Integrated Connectivity Protocols (WIFI, Bluetooth, BLE), On chip sensor (Hall sensor), Operating temperature (range between 40 − 85 degrees Celsius), Operating Voltage (3.3V), Operating Current (80 mA average).

With the above specifications in front of you, it is very easy to decipher the reasons for ESP32's popularity. Consider the requirements an IoT device would have from its microcontroller (μC). If you've gone through the previous chapter, you'd have realized that the major operational blocks of any IoT device are sensing, processing, storage, and transmitting. Therefore, to begin with, the μC should be able to interface with a variety of sensors. It should support all the common communication protocols required for sensor interface such as UART, I2C, SPI. It should have ADC and pulse counting capabilities. ESP32 fulfils all of these requirements. On top of that, it also can interface with capacitive touch sensors. Therefore, most common sensors can interface seamlessly with ESP32.

Secondly, the μC should be able to perform basic processing of the incoming sensor data, sometimes at high speeds, and have sufficient memory to store the data. ESP32 has a max operating frequency of 40 MHz, which is sufficiently high. It has two cores, allowing parallel processing, which is a further add-on. Finally, its 520 KB SRAM is sufficiently large for processing a large array of data onboard. Many popular processes and transforms, like FFT, peak detection, RMS calculation, etc. can be performed onboard ESP32. On the storage front, ESP32 goes a step ahead of the conventional microcontrollers and provides a file system within the flash. Out of the 4 MB of onboard flash, by default, 1.5 MB is reserved as SPIFFS (SPI Flash File System). Think of it as a mini−SD Card that lies within the chip itself. You can not only store data, but also text files, images, HTML and CSS files, and a lot more within SPIFFS. People have displayed beautiful Webpages on Wi-Fi servers created using ESP32, by storing HTML files within SPIFFS.

Finally, for transmitting data, ESP32 has integrated Wi-Fi and Bluetooth stacks, which have proven to be a game-changer. No need to connect a separate

module (like a GSM module or an LTE module) for testing cloud communication. Just have the ESP32 board and a running Wi-Fi, and you can get started. ESP32 allows you to use Wi-Fi in Access Point as well as Station Mode. While it supports TCP/IP, HTTP, MQTT, and other traditional communication protocols, it also supports HTTPS. Yep, you heard that right. It has a crypto−core or a crypto-accelerator, a dedicated piece of hardware whose job is to accelerate the encryption process. So, you cannot only communicate with your web server, you can do so securely. BLE support is also critical for several applications. Of course, you can interface LTE or GSM or LoRa modules with ESP32. Therefore, on the 'transmitting data' front as well, ESP32 exceeds expectations.

ESP32 is a chip that provides Wi-Fi and (in some models) Bluetooth connectivity for embedded devices like in other words, for IoT devices. While ESP32 is technically just the chip, the modules and development boards that contain this chip are often also referred to as “ESP32” by the manufacturer. The original ESP32 chip had a single core Tensilica Xtensa LX6 microprocessor. The processor had a clock rate of over 240 MHz, which made for a relatively high data processing speed.

More recently, new models were added, including the ESP32-C and -S series, which include both single and dual core variations. These two series also rely on a RISC-V CPU model instead of Xtensa. RISC-V is similar to the ARM architecture, which is well-supported and well-known, but RISC-V is open source and easy to use. Specifically, RISC-V and ARM have good support from GNU compilers, while the Xtensa needed extra support and development to work with the compilers.

The newer models are available with combined Wi-Fi and Bluetooth connectivity, or just Wi-Fi connectivity. There are several different chip models available, such as ESP32-D0WDQ6 (and ESP32D0WD), ESP32-D2WD, ESP32-S0WD, System in package (SiP) for (ESP32-PICO-D4), ESP32 S series, ESP32-C series, ESP32-H series.

The ESP32 is most commonly used for mobile devices, wearable tech, and IoT applications, such as Nabto Edge. Moreover, since Mongoose OS introduced an ESP32 IoT Starter Kit, the ESP32 has gained a reputation as the ultimate chip for hobbyists and IoT developers. It’s suitable for commercial IoT, and its capabilities and resources have grown impressively over the past four years.

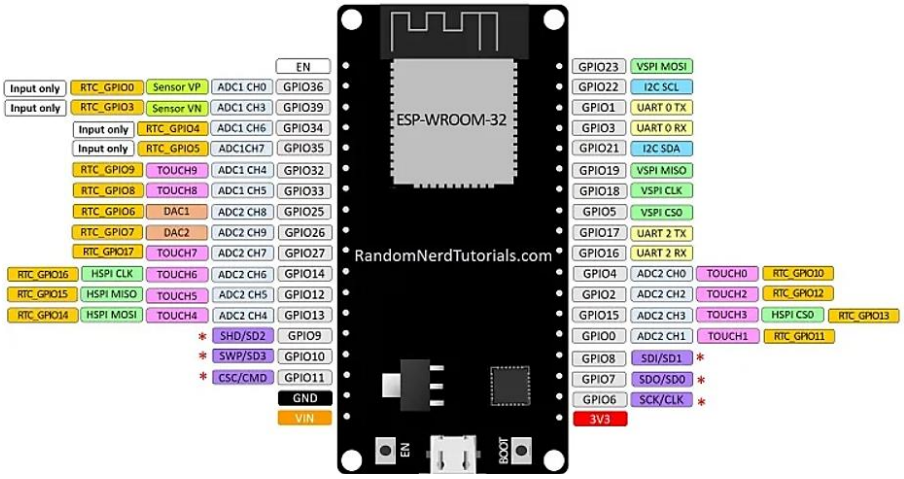

Fig. 6.2: ESP32 Pin Configuration

Here's a high-level summary of the features and specifications of the ESP32:

ESP-32	DESCRIPTION
Core	2
Architecture	32 bits
Clock	Tensilica Xtensa LX106 160-240MHz
WiFi	IEEE802.11 b/g/n
Bluetooth	Yes - classic & BLE
RAM	520KB
Flash	External QSPI - 16MB
GPIO	22
DAC	2
ADC	18
Interfaces	SPI-I2C-UART-I2S-CAN

Table 6.1: Features and Specifications of ESP32

Processors: The ESP32 uses a Tensilica Xtensa 32-bit LX6 microprocessor. This typically relies on a dual core architecture, with the exception of one module, the ESP32-S0WD, which uses a single-core system. The clock frequency reaches up to 240MHz and it performs up to 600 DMIPS (Dhrystone millions of instructions per second). Moreover, its low power consumption allows for analog to digital conversions as well as computation and level thresholds, even while the chip is in deep sleep mode.

Wireless connectivity: The ESP32 enables connectivity to integrated Wi-Fi through the 802.11 b/g/n/e/i/. Moreover, Bluetooth connectivity is made possible with the v4.2 BR/EDR, and the series also features Bluetooth low energy (BLE).

Memory: Internal memory for the ESP32 is as follows ROM (448 KB for booting/core functions), SRAM (520 KB for data/instructions), RTC fast SRAM (8 KB for data storage/main CPU during boot from sleep mode), RTC slow SRAM (8 KB for co-processor access during sleep mode), and eFuse (1 KiBit 256 bits used for the system (MAC address and chip configuration) and 768 bits reserved for customer applications). Moreover, some of the ESP32 chips, including the ESP32-D2WD and ESP32-PICO-D4, have internally connected flash. See the respective internal flash memory for each chip in the ESP32 Chips section.

External flash and SRAM: ESP32 supports up to four 16 MB external QSPI flashes and SRAMs with hardware encryption based on AES to protect developers' programs and data. It accesses the external QSPI flash and SRAM through high-speed caches.

Security: The ESP32 supports all IEEE 802.11 standard security features, including WFA, WPA/WPA2 and WAPI. Moreover, ESP32 has a secure boot and flash encryption.

6.1.2 ESP32 Functions

ESP32 has many functionalities when it comes to IoT. Here are just some of the IoT functions, the chip which is used the following are Networking, the module's Wi-Fi antenna and dual core enables embedded devices to connect to routers and transmit data. Data processing, includes processing basic inputs from analog and digital sensors to far more complex calculations with an RTOS or non-OS software development kit (SDK). A non-OS SDK refers to one that is designed to run directly on the chip without a full operating system supporting it. P2P connectivity, creates direct communication between different ESPs and other devices using IoT P2P connectivity. Web server, provides access to pages written in HTML or development languages.

6.1.3 Applications of ESP32

The ESP32 modules are commonly found in the following IoT devices are Smart industrial devices, including programmable logic controllers (PLCs). Smart medical devices, including wearable health monitors. Smart energy devices, including HVAC and thermostats. Smart security devices, including surveillance cameras and smart locks.

6.1.4 Chip versus modules versus development boards:

The ESP32 is just the name of the chip. Device manufacturers and developers have three different format choices, and the decision of which one to go with will depend on their individual circumstances:

ESP32 chip: This is the bare-bones chip that is manufactured by Espressif. It comes unshielded, meaning there's no protective casing, and it can't be attached to a module or board without soldering. Therefore, most device manufacturers do not purchase just the chip, as this will add an additional layer of complexity to the production process.

ESP32 modules: These are surface mountable modules that contain the chip. The modules are essentially small electrical components that can be attached to a circuit board. The benefit here is that you can easily mount these modules onto an MCU board. The chip is also usually shielded and pre-approved by the FCC, which means device manufacturers do not need to worry about adding additional steps to the production process to achieve FCC compliance regarding Wi-Fi shielding.

ESP32 development boards: These are IoT MCU development boards that have the modules containing the ESP32 chip preinstalled. They are used by hobbyists, device manufacturers and developers to test and prototype IoT devices before entering mass production. There is a wide variety of makes and models of ESP32 development boards, produced by different manufacturers.

Here are some important specs to consider when choosing a suitable IoT ESP32 development board such as GPIO pins, ADC pins, Wi-Fi antennas, LEDs, Shielding, Flash memory.

Many international markets require shielded Wi-Fi devices, as Wi-Fi produces a lot of radio frequency interference (RFI), and shielding minimizes this interference. This should, therefore, be a key consideration for all developers and embedded device manufacturers.

6.2 GAS SENSOR (MQ-2)

The MQ-2 Gas Sensor is a widely used electronic component for detecting combustible gases such as LPG, methane, butane, propane, hydrogen, alcohol, and smoke. It operates on the principle of metal oxide semiconductor (MOS) gas sensing, where the resistance of its tin dioxide (SnO_2) sensing layer changes in the presence

of gas, allowing for detection and measurement. This sensor is commonly used in gas leak detection systems, fire alarms, air quality monitoring, and industrial safety applications.

Fig. 6.3: MQ-2 Gas Sensor

6.2.1 Description

The MQ-2 sensor requires a 5V DC power supply and consumes around 150mA of current. It provides both analog (AO) and digital (DO) outputs, making it compatible with microcontrollers like Arduino, ESP32, and Raspberry Pi. The analog output gives a variable voltage depending on the gas concentration, while the digital output acts as a threshold-based trigger, which can be adjusted using an onboard potentiometer. The sensor has a detection range of 200 to 10,000 ppm and typically responds to gas presence within 10 seconds.

For reliable operation, the MQ-2 sensor requires a preheating time of about 20–30 seconds to stabilize its readings. It should be used in a well-ventilated environment and kept away from excessive moisture, dust, or oil vapours, as these can degrade its performance. When interfacing with microcontrollers, the sensor can be used to trigger alarms, buzzers, or automatic ventilation systems if a dangerous gas level is detected. Due to its affordability, high sensitivity, and ease of integration, the MQ-2 sensor is widely utilized in smart home safety systems, industrial monitoring, and IoT-based gas detection projects.

6.3 FLAME SENSOR

This module is sensitive to the flame and radiation. It also can detect ordinary light source in the range of a wavelength 760nm-1100 nm. The detection distance is up to 100 cm. The Flame sensor can output digital or analog signal. It can be used as

a flame alarm or in firefighting robots. Future Electronics Egypt Ltd. (Arduino Egypt).

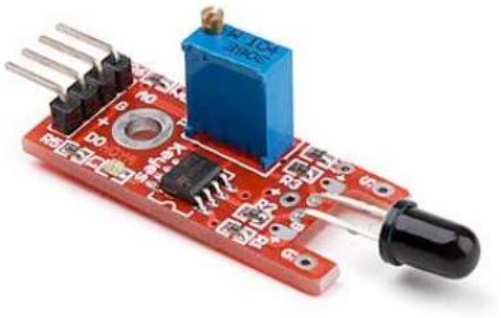

Fig. 6.4: Flame Sensor

6.3.1 Description

This module is designed to detect flames or light sources within the wavelength range of 760nm to 1100nm. It offers a detection distance ranging from 20cm at 4.8V to 100cm at 1V and has a detection angle of approximately 60 degrees, making it highly sensitive to the flame spectrum. The built-in LM393 comparator chip ensures stable readings, while the detection range is adjustable for better flexibility. It operates within a voltage range of 3.3V to 5V and provides both digital and analog output options. The digital output (DO) functions as a switch, providing binary outputs of 0 and 1, while the analog output (AO) delivers variable voltage readings. Additionally, the module features a power indicator and a digital switch output indicator for easy monitoring. The interface consists of four connections: VCC for 3.3V-5V power input, GND for ground, DO for digital output, and AO for analog output.

6.4 RTC (DS 1302)

The DS1302 trickle-charge timekeeping chip contains a real-time clock/calendar and 31 bytes of static RAM. It communicates with a microprocessor via a simple serial interface. The real-time clock/calendar provides seconds, minutes, hours, day, date, month, and year information. Only three wires are required to communicate with the clock/RAM: RST, I/O (data line), and SCLK (serial clock). Data can be transferred to and from the clock/RAM 1 byte at a time or in a burst of up to 31 bytes. The DS1302 is designed to operate on very low power and retain data and clock information on less than 1μW. The DS1302 has dual power pins, one for primary and another for backup.

Fig. 6.5: RTC Module

6.4.1 Features

This real-time clock module is capable of counting seconds, minutes, hours, the date of the month, month, day of the week, and year, with leap year compensation valid up to the year 2100. It includes a 31 x 8 RAM for scratchpad data storage and features a serial I/O interface that minimizes pin usage. The module operates efficiently within a voltage range of 2.0V to 5.5V, consuming less than 300 nA at 2.0V. It supports both single-byte and multiple-byte (burst mode) data transfer for reading or writing clock or RAM data. Available in an 8-pin DIP or optional 8-pin SOIC package for surface mounting, it features a simple 3-wire interface and is TTL-compatible when powered at 5V. The module offers an optional industrial temperature range from -40°C to +85°C and is DS1202 compatible, with additional features over the DS1202. It includes an optional trickle charge capability to VCC1 and dual power supply pins, allowing for both primary and backup power supplies. The backup power supply pin can be used with either a battery or a super capacitor input. Additionally, it provides extra scratchpad memory with seven additional bytes for improved functionality.

6.4.2 Description

The DS1302 Trickle Charge Timekeeping Chip contains a real time clock/calendar and 31 bytes of static RAM. It communicates with a microprocessor via a simple serial interface. The real time clock/calendar provides seconds, minutes, hours, day, date, month, and year information. The end of the month date is automatically adjusted for months with less than 31 days, including corrections for leap year. The clock operates in either the 24–hour or 12–hour format with an AM/PM indicator.

Interfacing the DS1302 with a microprocessor is simplified by using synchronous serial communication. Only three wires are required to communicate with the clock/RAM: (1) RST (Reset), (2) I/O (Data line), and (3) SCLK (Serial clock). Data can be transferred to and from the clock/RAM 1 byte at a time or in a burst of up to 31 bytes. The DS1302 is designed to operate on very low power and retain data and clock information on less than 1 microwatt. The DS1302 is the successor to the DS1202. In addition to the basic timekeeping functions of the DS1202, the DS1302 has the additional features of dual power pins for primary and back–up power supplies, programmable trickle charger for VCC1, and seven additional bytes of scratchpad memory.

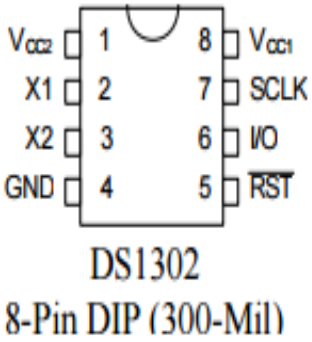

Fig. 6.6: Pin Assignment of RTC

6.4.3 Operation

The main elements of the Serial Timekeeper are shown in the above figure, shift register, control logic, oscillator, real time clock, and RAM. To initiate any transfer of data, RST is taken high and 8 bits are loaded into the shift register providing both address and command information. Data is serially input on the rising edge of the SCLK. The first 8 bits specify which of 40 bytes will be accessed, whether a read or write cycle will take place, and whether a byte or burst mode transfer is to occur. After the first eight clock cycles have loaded the command word into the shift register, additional clocks will output data for a read or input data for a write. The number of clock pulses equals 8 plus 8 for byte mode or 8 plus up to 248 for burst mode.

6.5 16x2 I2C LCD MODULE

A 16x2 I2C LCD module is a widely used display that simplifies interfacing with microcontrollers using the I2C (Inter-Integrated Circuit) protocol. It reduces the

number of required connections compared to a traditional 16x2 parallel LCD, making it ideal for embedded systems with limited GPIOs.

Fig. 6.7: LCD Display

6.5.1 Features

The I2C 16x2 LCD Module is a widely used display component featuring 16 columns and 2 rows for character display. It utilizes an I2C interface, which significantly reduces wiring complexity compared to parallel LCDs. The module is powered by the PCF8574 I2C GPIO expander, allowing seamless communication with microcontrollers. It supports backlight control, operates at 5V (with some versions supporting 3.3V), and is compatible with popular development boards such as Arduino, ESP32, STM32, and Raspberry Pi. Additionally, an adjustable potentiometer is included to fine-tune the display contrast for optimal readability.

6.5.2 Pin Diagram

The pin configuration of an I2C 16x2 LCD module typically consists of four pins: GND (Ground - 0V), VCC (Power Supply - usually 5V, but some versions support 3.3V), SDA (Serial Data Line for I2C communication), and SCL (Serial Clock Line for timing synchronization). These pins facilitate an efficient connection to microcontrollers while minimizing the number of required connections, making the module a preferred choice for compact and simple display setups.

6.5.3 Internal Components of I2C Module

Internally, the I2C LCD module consists of several key components. The PCF8574 I2C to Parallel Converter expands I2C communication to drive the HD44780 LCD controller, enabling parallel operation via I2C commands. The 16x2 LCD panel is based on the Hitachi HD44780 controller, ensuring compatibility with standard libraries and software. An adjustable potentiometer is integrated into the

module for contrast adjustment, allowing users to modify text visibility. Additionally, the backlight LED control can be toggled on or off via software, providing better visibility in low-light environments.

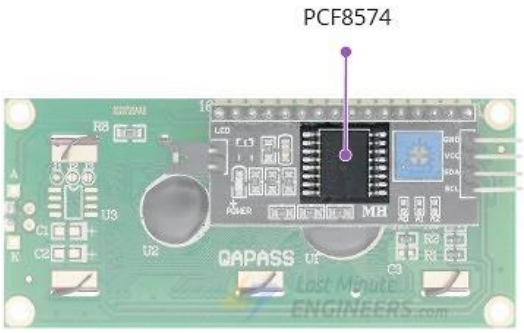

Fig. 6.8: PCF8574 I2C in LCD module

The board also includes a tiny trimpot for making precise adjustments to the display's contrast. There is a jumper on the board that provides power to the backlight. To control the intensity of the backlight, you can remove the jumper and apply external voltage to the header pin labelled 'LED'.

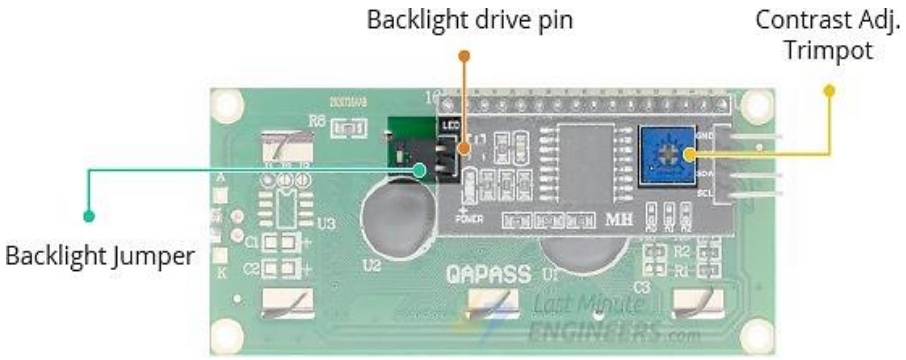

Fig. 6.9: Backlight & Trimpot in PCF8574

6.6 DVD LOADER DC MOTOR

A DVD loader DC motor is a compact brushed DC motor used in DVD/CD drives to control the movement of the disc tray (loader mechanism). It operates at low voltage (typically 5V - 12V) and provides precise rotational motion for opening and closing the tray.

A dc motor uses electrical energy to produce mechanical energy, very typically through the interaction of magnetic fields and current-carrying conductors. The reverse process, producing electrical energy from mechanical energy, is accomplished by an alternator, generator or dynamo. Many types of electric motors

can be run as generators, and vice versa. The input of a DC motor is current/voltage and its output is torque (speed).

Fig. 6.10: DC Motor

6.6.1 Types of Motors in a DVD Drive

A DVD drive typically consists of three different motors, each serving a specific function. The spindle motor is responsible for spinning the DVD or CD disc, ensuring stable rotation for data reading and writing. The sled motor, which can be either a stepper or DC motor, moves the laser assembly precisely across the disc surface to read or write data. Lastly, the loader motor, also known as the tray eject motor, controls the opening and closing of the DVD tray, allowing for easy insertion and removal of discs. These motors work together to ensure smooth operation and efficient performance of the DVD drive.

Parameter	Details
Type	Brushed DC Motor
Operating Voltage	5V - 12V DC
Current Rating	100mA - 500mA
Speed	~1000 - 5000 RPM
Torque	Low (suitable for lightweight tray movement)
Size	Small (typically cylindrical or rectangular)
Drive Mechanism	Connected to gears & belt to drive the tray

Table 6.2: Specifications of DVD Loader Motor

The polarity of the voltage applied to the motor determines the direction of the tray movement in a DVD drive. When normal polarity is applied, with VCC connected to the positive terminal and GND to the negative terminal, the tray ejects or opens. Conversely, when the polarity is reversed, with VCC connected to the negative terminal and GND to the positive terminal, the tray retracts or closes. This bidirectional control allows for smooth and precise operation of the DVD tray mechanism.

6.6.2 Working Mechanism

The DC motor in a DVD drive is connected to a gear and belt system that facilitates the movement of the tray. When powered, the motor rotates, driving the belt to push or pull the DVD tray as needed. The direction of the tray movement is controlled by reversing the polarity, which can be achieved using an H-Bridge circuit or a relay. To ensure precise operation, a limit switch is used to detect when the tray is fully opened or closed, automatically stopping the motor to prevent overextension or damage.

Fig. 6.11: DVD Loader Driver

6.7 VOICE MODULE (APR33A3)

The APR33A3 series are powerful audio processor along with high performance audio analog-to-digital converters (ADCs) and digital-to-analog converters (DACs). They are a fully integrated solution offering high performance and unparalleled integration with analog input, digital processing and analog output functionality.

6.7.1 Description

The APR33A3 series incorporates all the functionality required to perform demanding audio/voice applications. High quality audio/voice systems with lower bill-of-material costs can be implemented with the APR33A3 series because of its integrated analog data converters and full suite of quality-enhancing features such as sample-rate convertor.

The APR33A series C2.0 is specially designed for simple key trigger, user can record and playback the message averagely for 1, 2, 4 or 8 voice message(s) by switch, it is suitable in simple interface or need to limit the length of single message,

e.g. toys, leave messages system, answering machine etc. Meanwhile, this mode provides the power-management system. Users can let the chip enter power-down mode when unused. It can effectively reduce electric current consuming to 15uA and increase the using time in any projects powered by batteries.

Fig. 6.12: APR33A3 Module

6.8 DC MOTOR ROBOT

A robot using DC motors is commonly known as a mobile robot or wheeled robot. The DC motors, often combined with gearboxes, control the movement of the robot by driving its wheels or actuators. The direction, speed, and movement of the robot are controlled using a microcontroller along with a motor driver (like L298N or L293D)

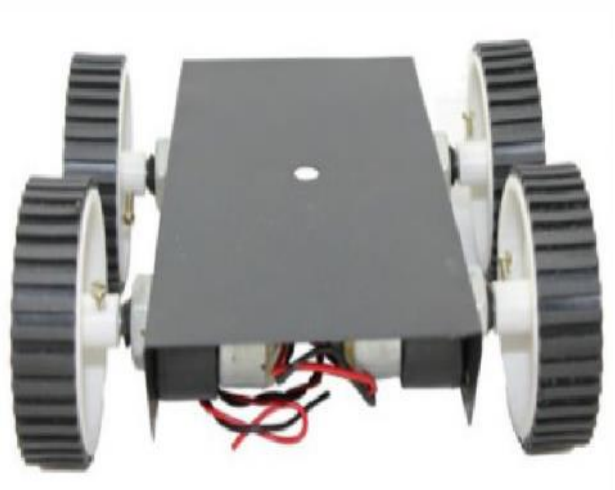

Fig. 6.13: DC Motor Robot

6.8.1 Basic Components

The robot's movement is facilitated by DC motors with wheels, allowing it to move forward, backward, left, and right with precision. These motors are controlled

by a motor driver, such as the L298N or L293D, which regulates their speed and direction based on commands from the microcontroller. The microcontroller serves as the brain of the system, sending control signals to the motor driver to ensure smooth operation.

To power the motors and electronic components, the robot is equipped with a battery ranging from 6V to 12V, providing a reliable power supply. All components are securely mounted on a chassis, which acts as the frame of the robot, ensuring structural stability. Additionally, optional sensors like ultrasonic, infrared, or line sensors can be integrated for autonomous movement, enabling the robot to navigate and respond to its environment efficiently.

6.8.2 Working

A basic robotic vehicle consists of several key components that work together to enable movement and control. DC motors with wheels drive the robot in different directions, including forward, backward, left, and right. These motors are controlled by a motor driver, such as the L298N or L293D, which regulates both speed and direction. Since microcontrollers cannot directly drive DC motors due to high current requirements, a motor driver is essential for proper operation. The microcontroller sends necessary control signals to the motor driver, ensuring smooth movement. The entire system is powered by a battery pack, typically ranging from 6V to 12V, supplying energy to both the motors and other electronic components. All these elements are securely mounted on a chassis or frame, providing structure and stability to the robot. Additionally, optional sensors like ultrasonic, infrared, or line sensors can be integrated for autonomous navigation, allowing the robot to move and respond intelligently to its environment.

The movement of the robot is controlled through an H-Bridge circuit in the motor driver, which allows bidirectional movement by reversing the polarity of the motor. The microcontroller sends signals to adjust motor direction and speed. When both motors rotate in the same direction, the robot moves forward, while reversing their direction makes it move backward. To turn left or right, the robot either rotates only one motor or moves the motors in opposite directions.

A DC motor operates by converting electrical energy into mechanical energy through the interaction of magnetic fields and current-carrying conductors. The reverse process, generating electrical energy from mechanical energy, is

accomplished by devices like alternators, generators, or dynamos. Many electric motors can function as generators and vice versa. The input to a DC motor is electrical current or voltage, and its output is mechanical torque and speed, making it a fundamental component in various electromechanical systems.

Motor 1	Motor 2	Robot Movement
Forward	Forward	Moves Forward
Backward	Backward	Moves Backward
Forward	Stop	Turns Right
Stop	Forward	Turns Left

Table 6.3: Working of DC Motor Robot

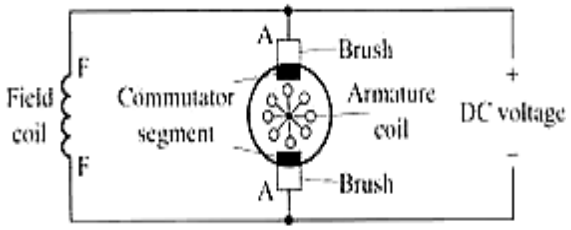

Fig. 6.14: Simple electrical diagram of DC motor

The DC motor has two basic parts: the rotating part that is called the armature and the stationary part that includes coils of wire called the field coils. The stationary part is also called the stator. Figure shows a picture of a typical DC motor, Figure shows a picture of a DC armature, and Fig shows a picture of a typical stator. From the picture you can see the armature is made of coils of wire wrapped around the core, and the core has an extended shaft that rotates on bearings. You should also notice that the ends of each coil of wire on the armature are terminated at one end of the armature. The termination points are called the commutator, and this is where the brushes make electrical contact to bring electrical current from the stationary part to the rotating part of the machine.

6.8.3 Operation

The DC motor you will find in modem industrial applications operates very similarly to the simple DC motor described earlier in this chapter. Figure 12-9 shows an electrical diagram of a simple DC motor. Notice that the DC voltage is applied directly to the field winding and the brushes. The armature and the field are both

shown as a coil of wire. In later diagrams, a field resistor will be added in series with the field to control the motor speed.

When voltage is applied to the motor, current begins to flow through the field coil from the negative terminal to the positive terminal. This sets up a strong magnetic field in the field winding. Current also begins to flow through the brushes into a commutator segment and then through an armature coil. The current continues to flow through the coil back to the brush that is attached to other end of the coil and returns to the DC power source. The current flowing in the armature coil sets up a strong magnetic field in the armature.

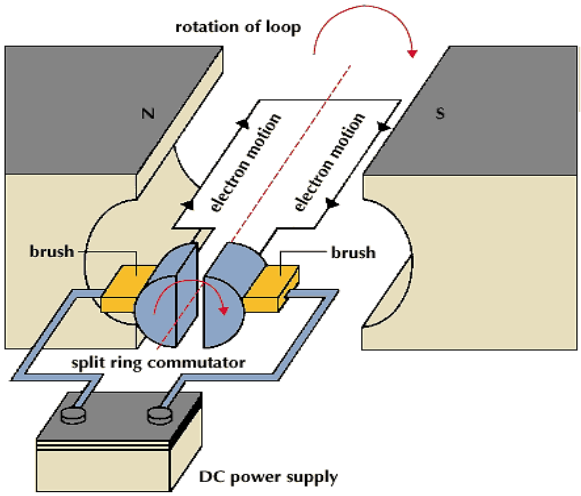

Fig. 6.15: Operation of DC Motor

The magnetic field in the armature and field coil causes the armature to begin to rotate. This occurs by the unlike magnetic poles attracting each other and the like magnetic poles repelling each other. As the armature begins to rotate, the commutator segments will also begin to move under the brushes. As an individual commutator segment moves under the brush connected to positive voltage, it will become positive, and when it moves under a brush connected to negative voltage it will become negative. In this way, the commutator segments continually change polarity from positive to negative. Since the commutator segments are connected to the ends of the wires that make up the field winding in the armature, it causes the magnetic field in the armature to change polarity continually from North Pole to South Pole. The commutator segments and brushes are aligned in such a way that the switch in polarity of the armature coincides with the location of the armature's magnetic field and the field winding's magnetic field. The switching action is timed so that the armature will not lock up magnetically with the field. Instead, the magnetic

fields tend to build on each other and provide additional torque to keep the motor shaft rotating.

When the voltage is de-energized to the motor, the magnetic fields in the armature and the field winding will quickly diminish and the armature shaft's speed will begin to drop to zero. If voltage is applied to the motor again, the magnetic fields will strengthen and the armature will begin to rotate again.

6.9 L293D DC MOTOR DRIVER

The L293 and L293D are quadruple high-current half-H drivers. The L293 is designed to provide bidirectional drive currents of up to 1 A at voltages from 4.5 V to 36 V. The L293D is designed to provide bidirectional drive currents of up to 600-mA at voltages from 4.5 V to 36 V. Both devices are designed to drive inductive loads such as relays, solenoids, dc and bipolar stepping motors, as well as other high-current/high-voltage loads in positive-supply applications.

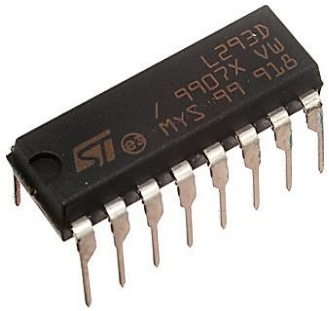

Fig. 6.16: L293D IC

All inputs are TTL compatible. Each output is a complete totem-pole drive circuit, with a Darlington transistor sink and a pseudo-Darlington source. Drivers are enabled in pairs, with drivers 1 and 2 enabled by 1,2EN and drivers 3 and 4 enabled by 3, 4EN.When an enable input is high, the associated drivers are enabled and their outputs are active and in phase with their inputs.

When the enable input is low, those drivers are disabled and their outputs are off and in the high-impedance state. With the proper data inputs, each pair of drivers forms a full-H (or bridge) reversible drive suitable for solenoid or motor applications. On the L293, external high-speed output clamp diodes should be used for inductive transient suppression. A VCC1 terminal, separate from VCC2, is provided for the logic inputs to minimize device power dissipation. The L293and L293D are characterized for operation from 0°C to 70°C.

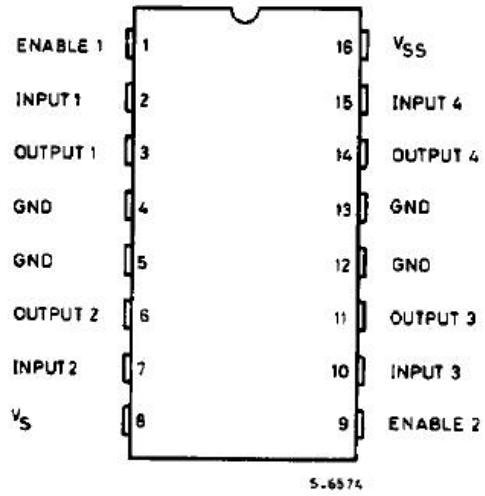

Fig. 6.17: Pin Diagram of L293D Motor Driver

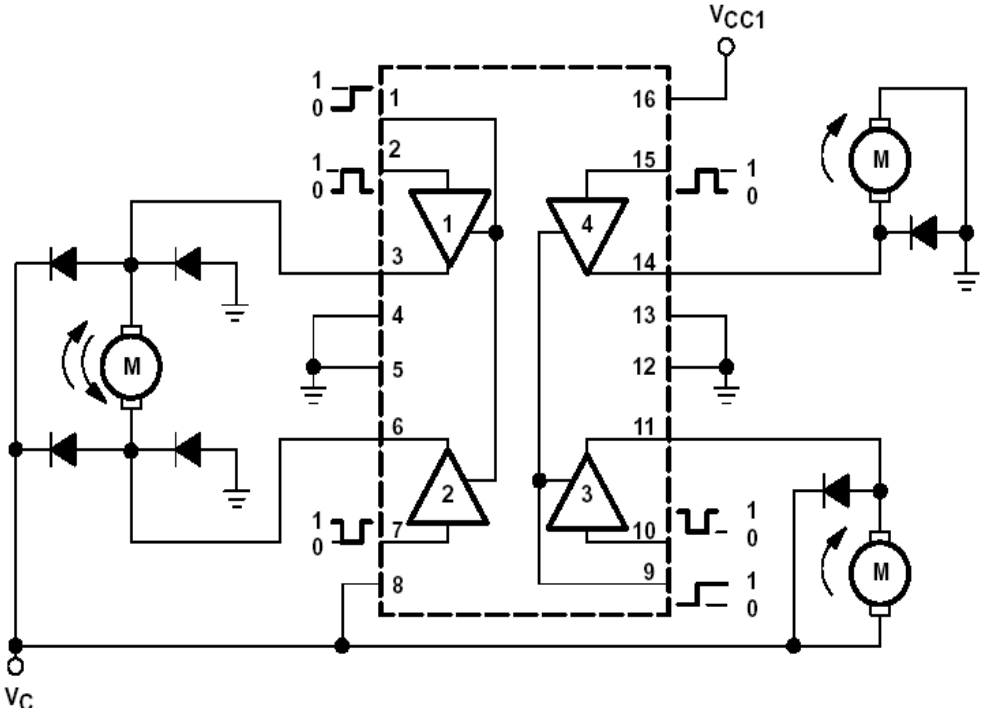

Fig. 6.18: Internal structure of L293D

6.9.1 Features of L293D

The motor driver offers an output current capability of 600mA per channel, with a peak output current of up to 1.2A per channel for non-repetitive operations. It includes an enable facility, allowing precise control over motor operation. The driver is equipped with over-temperature protection, ensuring safe and reliable performance by preventing overheating. It also features a logical "0" input voltage tolerance of up to 1.5V, providing compatibility with various control systems. With high noise immunity, the driver ensures stable operation even in electrically noisy

environments. Additionally, internal clamp diodes are integrated to protect against voltage spikes, enhancing the durability and efficiency of the circuit.

6.9.2 Applications of DC Motors

DC motors are widely used in various applications due to their efficiency, precision, and adaptability. In electric trains, DC series motors are commonly used because they have the unique ability to deliver more power as the load increases. This means that as more passengers board the train, the motor generates more power, ensuring smooth operation. Similarly, elevators rely on DC motors for their bidirectional functionality, with compound DC motors being the preferred choice for providing stable and efficient movement.

In smaller-scale applications, DC motors are essential components in PC fans, CD-ROM drives, and hard drives. These devices require miniature motors with high precision, a function that AC motors cannot fulfill. Additionally, starter motors in automobiles utilize DC motors because car batteries supply direct current, making them the ideal choice for starting engines. Unlike AC motors, DC motors provide the necessary torque and reliability required for this application. Lastly, DC motors play a crucial role in electrical machines laboratories in colleges, where they are used for educational and experimental purposes, helping students understand motor functionality and applications in real-world scenarios.

6.10 IP CAMERA

An IP camera (Internet Protocol camera) is a type of digital video camera used for surveillance and security systems. Unlike traditional analog CCTV cameras, IP cameras transmit video data over a network (Wi-Fi or Ethernet) and can be accessed remotely using a smartphone, computer, or cloud service.

6.10.1 Key Features of IP Camera

The features of IP Cameras are the following such as Digital video transmission over Wi-Fi or Ethernet, High-resolution recording (720p, 1080p, 4K), Remote access via Smartphone apps or web browsers, Motion detection & alerts, Night vision (IR LEDs for low-light surveillance), Two-way audio (microphone & speaker), Cloud storage or local recording (SD card/NVR).

6.10.2 Working Principle

An IP Camera operates through a series of steps that ensure efficient video capture, processing, transmission, and storage. The process begins with image capture, where the camera records video using a CMOS or CCD image sensor. This raw footage is then processed by a built-in Digital Signal Processor (DSP), which compresses the video using formats such as H.264, H.265, or MJPEG. After processing, the camera transmits the video through wired **(**Ethernet) or wireless (Wi-Fi) networks to a router, enabling remote access. Users can view live or recorded footage via mobile apps, web browsers, or NVR (Network Video Recorder) software. Storage options for video recordings include SD cards, cloud storage, or NVRs, and many IP cameras feature motion detection, which triggers alerts such as email notifications, push messages, or alarms.

Fig. 6.19: IP Camera

6.10.3 IP Camera Pin Diagram / Interfaces

Most IP cameras feature multiple ports and interfaces that facilitate their functionality. The power input (DC 12V or 5V) supplies electricity to the device, while an Ethernet (RJ45) port allows for wired network connections. Wireless models include a Wi-Fi module for network connectivity. Many cameras also support local storage via a MicroSD slot, enabling onboard recording. Other essential interfaces include a reset button for factory resets, **a**udio input and output for two-way communication, and infrared (IR) LED**s** to enhance night vision capabilities.

Type	Description
Wired IP Camera	Uses Ethernet cable for stable connection.
Wireless IP Camera	Connects via Wi-Fi; easy installation.
PTZ IP Camera	Supports Pan-Tilt-Zoom for wide coverage.
Dome IP Camera	Compact design for indoor/outdoor use.
Bullet IP Camera	Cylindrical shape for long-range monitoring.
AI-Powered IP Camera	Uses AI for facial recognition & smart tracking.

Table 6.4: Types of IP Camera

6.10.4 Applications

IP cameras have a wide range of applications in different industries. They are commonly used in home security and smart home systems, providing real-time monitoring and motion alerts. Businesses and offices utilize them for surveillance and employee safety. In public areas, they assist in traffic monitoring and public safety by helping law enforcement manage incidents. Industrial facilities and warehouses benefit from enhanced security through constant monitoring of premises. Additionally, IP cameras serve as baby monitors and pet cameras, allowing caregivers to check on their loved ones remotely. These versatile applications make IP cameras an essential part of modern security solutions.

6.11 BUZZER

A piezoelectric buzzer produces sound using a piezoelectric diaphragm, which bends when AC voltage is applied, generating sound waves. A transistor circuit is typically used for interfacing, ensuring a common ground if a separate power supply is used.

Fig. 6.20: Buzzer

Piezo sounders consume less current than buzzers and can produce multiple tones, unlike single-tone buzzers. To switch the buzzer on, set HIGH (1); to switch it off, set LOW (0).

6.11.1 Handling Considerations

Improper switching may cause continuous sound due to residual feedback voltage. Follow recommended circuit designs for stable oscillation. Use direct power switching. Self-drive circuit is built-in; no external driver is needed. Maintain rated voltage (3-20V DC); higher voltage models are available. Avoid series resistors; if necessary, add a parallel capacitor (~1μF). Keep the sound-emitting hole unobstructed. Ensure no obstacles within 15mm of the sound release hole.

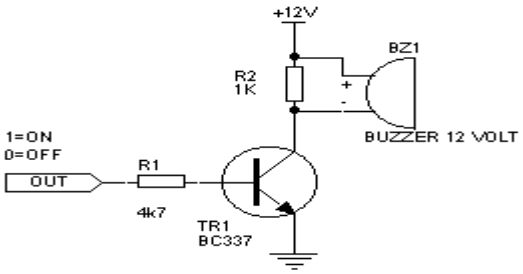

Fig. 6.21: Circuit diagram of Buzzer

6.11 BATTERY-POWER SUPPLY

A battery is a type of linear power supply that offers benefits that traditional line-operated power supplies lack: mobility, portability and reliability. A battery consists of multiple electrochemical cells connected to provide the voltage desired.

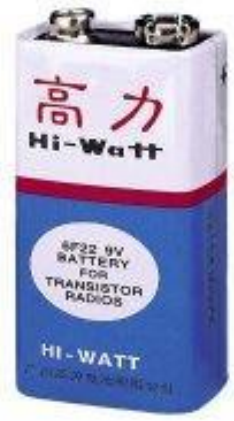

Fig. 6.22: Battery

The most commonly used dry-cell battery is the carbon-zinc dry cell battery. Dry-cell batteries are made by stacking a carbon plate, a layer of electrolyte paste, and a zinc plate alternately until the desired total voltage is achieved. The most common dry-cell batteries have one of the following voltages: 1.5, 3, 6, 9, 22.5, 45, and 90. During the discharge of a carbon zinc battery, the zinc metal is converted to

a zinc salt in the electrolyte, and magnesium dioxide is reduced at the carbon electrode. These actions establish a voltage of approximately 1.5 V. Rectification: The process of converting an alternating current to a pulsating direct current is called as rectification. For rectification purpose we use rectifiers.

Rectifiers: A rectifier is an electrical device that converts alternating current (AC) to direct current (DC), a process known as rectification. Rectifiers have many uses including as components of power supplies and as detectors of radio signals. Rectifiers may be made of solid-state diodes, vacuum tube diodes, mercury arc valves, and other components. A device that it can perform the opposite function (converting DC to AC) is known as an inverter. When only one diode is used to rectify AC (by blocking the negative or positive portion of the waveform), the difference between the term diode and the term rectifier is merely one of usage, i.e., the term rectifier describes a diode that is being used to convert AC to DC. Almost all rectifiers comprise a number of diodes in a specific arrangement for more efficiently converting AC to DC than is possible with only one diode. Before the development of silicon semiconductor rectifiers, vacuum tube diodes and copper (I) oxide or selenium rectifier stacks were used.

Bridge full wave rectifier: The rectifier which converts an ac voltage to dc voltage using both half cycles of the input ac voltage. The circuit has four diodes connected to form a bridge. The ac input voltage is applied to the diagonally opposite ends of the bridge. The load resistance is connected between the other two ends of the bridge. For the positive half cycle of the input ac voltage, diodes D1 and D3 conduct, whereas diodes D2 and D4 remain in the OFF state. The conducting diodes will be in series with the load resistance R_L and hence the load current flows through R_L. For the negative half cycle of the input ac voltage, diodes D2 and D4 conduct whereas, D1 and D3 remain OFF. The conducting diodes D2 and D4 will be in series with the load resistance R_L and hence the current flows through R_L in the same direction as in the previous half cycle. Thus, a bi-directional wave is converted into a unidirectional wave.

CHAPTER 7

SOFTWARE DESCRIPTION

The Arduino Software (IDE) makes it easy to write code and upload it to the board offline. We recommend it for users with poor or no internet connection. This software can be used with any Arduino board. here are currently two versions of the Arduino IDE, one is the IDE 2.0.0.

7.1 ARDUINO IDE – COMPILER

Here are currently two versions of the Arduino IDE, one is the IDE 1.x.x and the other is IDE 2.x. The IDE 2.x is new major release that is faster and even more powerful to the IDE 1.x.x. In addition to a more modern editor and a more responsive interface it includes advanced features to help users with their coding and debugging. The following can guide you with using the offline IDE (you can choose either IDE 1.x.x or IDE 2.x):

To get started with Arduino, first, download and install the Arduino Software (IDE). You can choose between Arduino IDE 1.x.x, which is available for Windows, Mac OS, Linux, Portable IDE for Windows and Linux, and ChromeOS, or the newer Arduino IDE 2.x. Once installed, connect your Arduino board to your computer using a USB cable. Next, open the Arduino Software (IDE), which serves as the Integrated Development Environment for writing, uploading, and communicating with Arduino boards. Programs created in the Arduino IDE are called sketches and are written in a built-in text editor. These sketches are saved with the .ino file extension, allowing easy editing and compilation. If you're using the offline Arduino IDE 1.x.x, you can efficiently develop and upload your projects without needing an active internet connection.

The Arduino IDE editor consists of four main areas. At the top, the Toolbar provides buttons for common functions, such as verifying and uploading programs, creating, opening, and saving sketches, and accessing the Serial Monitor. Below it, the Message Area gives feedback when saving or exporting sketches and displays errors if any occur. The central part of the editor is the Text Editor, where users write their code. At the bottom, the Text Console displays output generated by the Arduino Software (IDE), including complete error messages and other important information during the compilation and upload process. These components work together to

provide an efficient environment for programming and debugging Arduino projects. The bottom right-hand corner of the window displays the configured board and serial port.

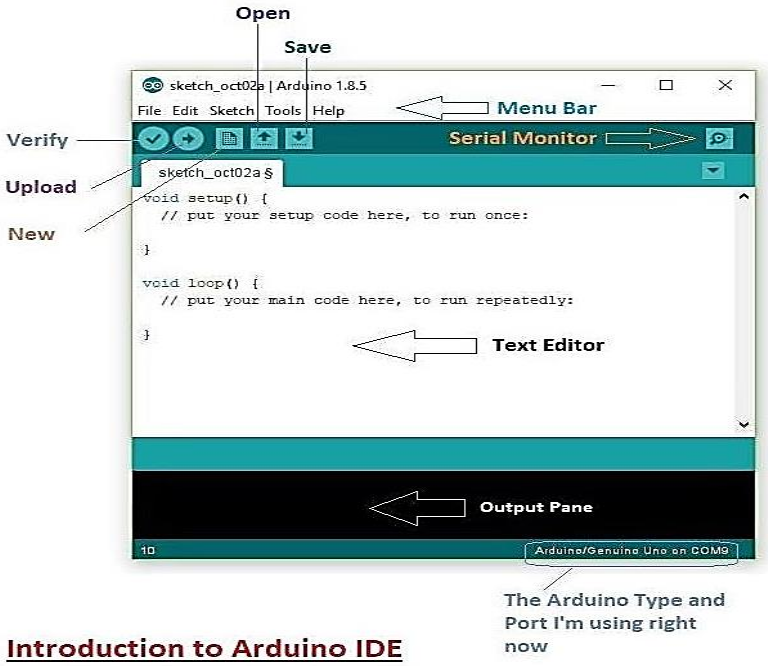

Fig. 7.1: Arduino Integrated Development Environment (IDE) Interface

Now that your Arduino Software (IDE) is set up, you can start by making your board blink. First, connect your Arduino or Genuino board to your computer using a USB cable. Next, you need to select the correct core and board in the Arduino IDE. To do this, navigate to Tools > Board > Arduino AVR Boards > Board and choose the board you are using. If your board is not listed, you can add it manually by going to Tools > Board > Boards Manager and installing the necessary board package. Once selected, your Arduino is ready for programming and testing.

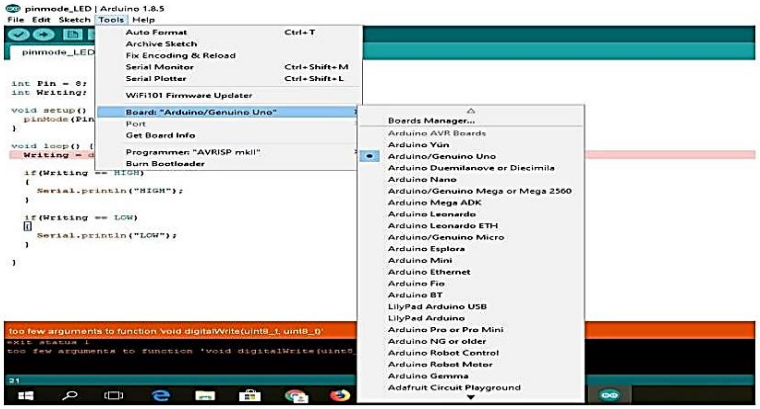

Fig. 7.2: Board menu option in Arduino IDE

To ensure that your Arduino board is properly detected by your computer, you need to select the correct port. This can be done by navigating to Tools > Port in the Arduino IDE, where you will see a list of available ports. Simply select the port associated with your Arduino board, allowing the IDE to establish a connection for uploading and communication.

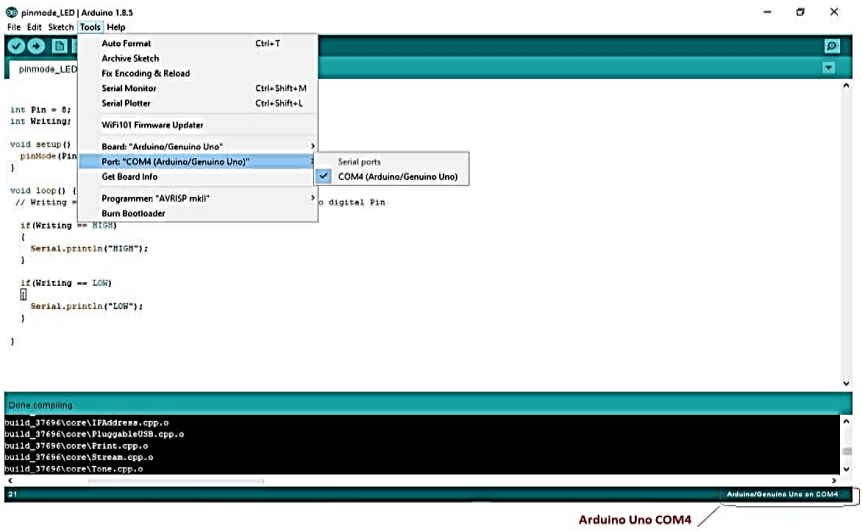

Fig. 7.3: Port menu option in Arduino IDE

To test your Arduino board, you can try a simple example by loading the Blink sketch. In the Arduino IDE, navigate to File > Examples > 01. Basics > Blink. This example program will make the built-in LED on your board blink, helping you verify that everything is set up correctly and functioning as expected.

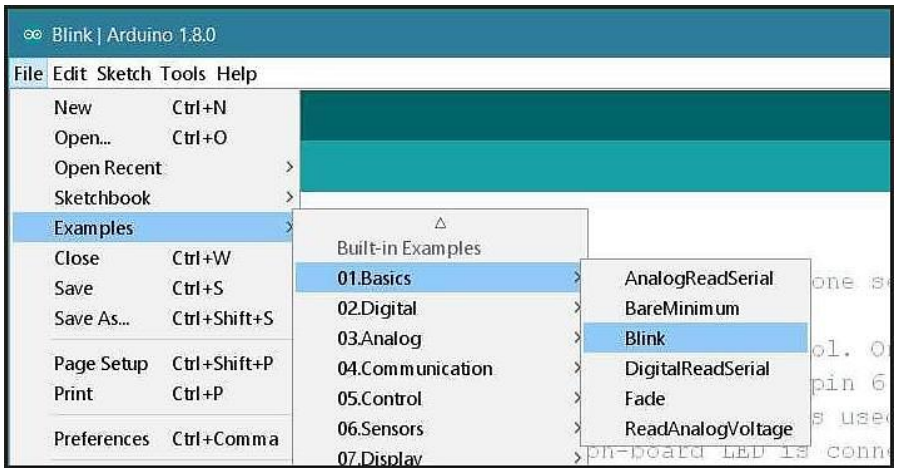

Figure 7.4: Basic examples to Get started with Arduino.

To upload the Blink sketch to your Arduino board, simply click the arrow button in the top-left corner of the Arduino IDE. The upload process takes a few seconds, and it is crucial not to disconnect the board during this time. If the upload

is successful, the message "Done uploading" will appear in the output area at the bottom. Once complete, the yellow LED labelled 'L' on your board should start blinking. You can adjust the blinking speed by changing the delay value in the parentheses to 100, then uploading the sketch again, this will make the LED blink faster.

The Arduino IDE editor consists of several key areas. At the top, the toolbar provides buttons for common functions such as verifying and uploading programs, managing sketches, selecting the board and port, and opening the Serial Monitor. The Sidebar offers quick access to essential tools, including the Board Manager, Library Manager, debugging tools, and a search-and-replace feature. The Text Editor is where you write and modify your code. Below, the Console Controls allow interaction with the console output, while the Text Console displays messages from the Arduino Software (IDE), including error messages and other relevant information. Finally, in the bottom-right corner of the window, you can see the currently configured board and serial port, ensuring that the correct device is selected for programming.

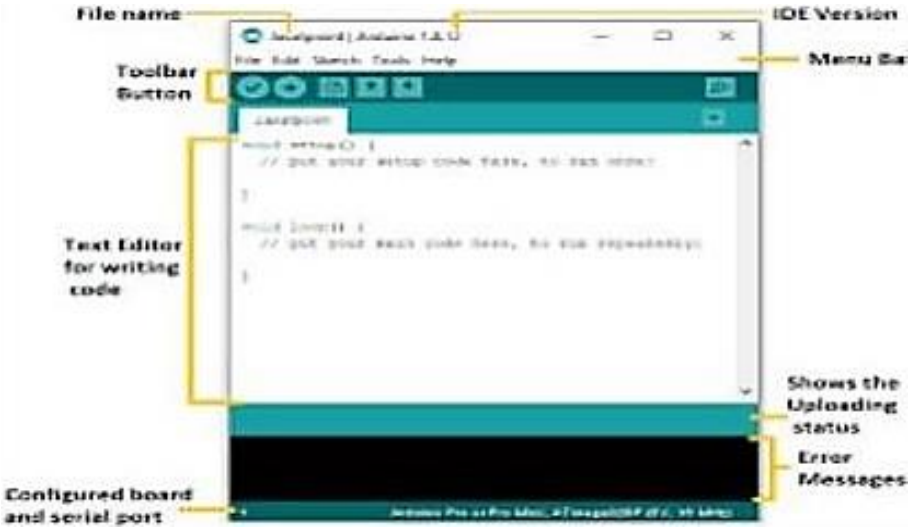

Fig. 7.5: Programming environment used for Software development

Now that your Arduino Software (IDE) is set up, you can begin by making your board blink. First, connect your Arduino or Genuino board to your computer using a USB cable. Next, you need to select the correct board and port from the toolbar in the Arduino IDE. Ensure that you choose the board you are using. If your board is not listed, you can add it manually by accessing the Board Manager in the sidebar. This step ensures that your IDE can communicate with the board properly,

allowing you to upload and run programs with the board properly, allowing you to upload and run programs.

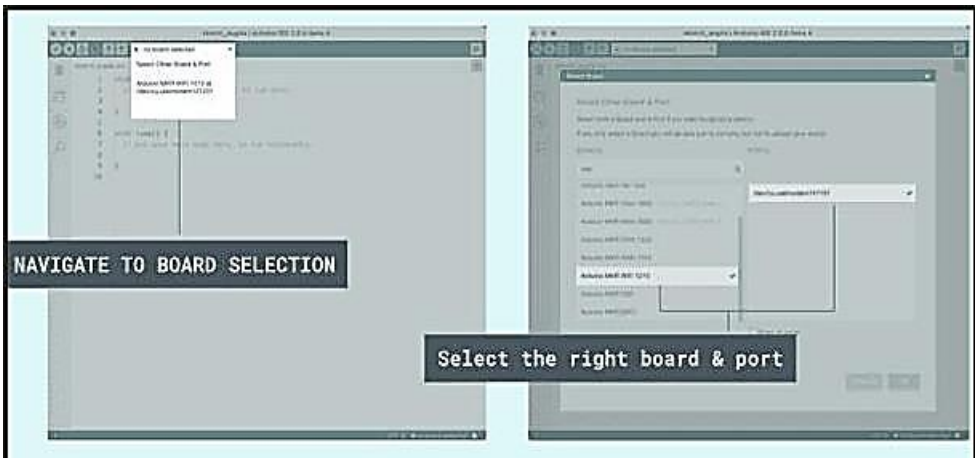

Fig. 7.6: Displaying instructions for Selecting a board and port

To upload the program to your Arduino board, simply click the arrow button in the top left corner of the Arduino IDE. The upload process takes a few seconds, and it is important not to disconnect the board during this time. Once the upload is complete, the message "Done uploading" will appear in the bottom output area, indicating that the program has been successfully transferred to the board.

CHAPTER 8

SOURCE CODE

```
#include <WiFi.h>
#include <HTTPClient.h>
#include <ArduinoJson.h>
#include <SimpleTimer.h>
SimpleTimer timecount;
SimpleTimer matchtimer;
const char *ssid = "project";  // replace with your wifi ssid and wpa2 key
const char *password = "project1234";
String get_host = "http://microembeddedtech.com/appinventor";
WiFiServer server(80);  // open port 80 for server connection
String tablename = "mediguardbot";
WiFiClient client;
#include <LiquidCrystal_I2C.h>LiquidCrystal_I2C lcd(0x27,16,2);
int dbupdate=0
int box1m1=2; int box2m1=15;
#define b1s1voicepin 26
#define b1s2voicepin 27
int MA1 = 21; int MB1 = 18;
int buzzerPin=23;
const char* d1s1time = "d1s1time";
const char* d2s2time = "d2s2time";
const char* robocmd = "robocmd";
const char* modebit = "modebit";
const char* d1manbit = "d1manbit";
const char* d2manbit = "d2manbit";
String k1, k2, k3, k4, k5, k6, k7, k8, k9, k10;
String currentime;
String chour; String cmin;
int p1bhour=0; int p1bmin=0;
int currenthour=1; int currentmin=1;
String b1s1time; String b2s2time;
```

```
String b1s1hour; String b1s1min;
int b1match = 0, b2match = 0, b3match = 0;
int robobit=5;
int modeval=0;
int box1manual=0; int b1manstatus=0;
int firepin=17;
String firestatus="NO";
int fireval;
int gaspin=16;
String gasstatus="NORMAL";
pinMode(buzzerPin, OUTPUT);
pinMode(firepin,INPUT);
 pinMode(gaspin,INPUT);
 digitalWrite(buzzerPin, 0);
  timer.setInterval(2000, timematchfunction);
  scantimer.setInterval(2000, scandata);
  lcd.begin(); lcd.clear();
  lcd.setCursor(0,0);
  lcd.print("IoT MEDIGUARD");
  lcd.setCursor(0,1);
  lcd.print("  BOT SYSTEM");
  robopins_setup();
  Serial.println(ssid);
}
  stoprobo();
  getdata();
}
void loop() {
  timer.run();
  getdata();
  delay(2000);
}
void scandata(){
  fireval=digitalRead(firepin);
```

```
  if(fireval==1){
    firestatus="NO";
  }
  else{
    firestatus="YES";
    buzzering();
  }
gasval=digitalRead(gaspin);
  if(gasval==1){
    gasstatus="NORMAL";
  }
  else{
    gasstatus="DETECTED";
    buzzering();
  }
 lcd.clear();
  lcd.setCursor(0,0);
  lcd.print("F:");
  lcd.print(firestatus);
  lcd.print("  ");
  lcd.print("G:");
  lcd.print(gasstatus);
  lcd.setCursor(0, 1);
  lcd.print("   ");
  lcd.print(currenthour); lcd.print(":"); lcd.print(currentmin); lcd.print("   ");
  userupdate_status(tablename,firestatus,gasstatus);
}
 void buzzering(){
  digitalWrite(buzzerPin, 1);
  delay(1000);
  digitalWrite(buzzerPin, 0);
}
void box1open(){
  Serial.println("-------- BOX 1 OPEN ----------");
```

```
  digitalWrite(box1m1,1);
  digitalWrite(box1m2,0);
  delay(1500);
  digitalWrite(box1m1,0);
  digitalWrite(box1m2,0);
  b1s1voice();
}
  void box1close(){
   Serial.println("-------- BOX 1 CLOSE ----------");
  digitalWrite(box1m1,0);
  digitalWrite(box1m2,1);
  delay(1500);
  digitalWrite(box1m1,0);
  digitalWrite(box1m2,0);
}
void get_device_status(String table_name) {
    if (WiFi.status() != WL_CONNECTED) {
        Serial.println("WiFi not connected! Retrying...");
        WiFi.begin(ssid, password);
        delay(3000);
        return;
    }
}
void checkstatus(){
b1s1time=k1;
b2s2time=k5;
robobit=k7.toInt();
modeval=k8.toInt();
box1manual=k9.toInt();
box2manual=k10.toInt();
  if(robobit==1){
  forwardrobo();
  }
  else if(robobit==2){
```

```
    backwardrobo() ;
  }
    else if(robobit==3){
    rightrobo();
  }
    else if(robobit==4){
    leftrobo();
  }
    else if(robobit==5){
    stoprobo();
  }
if(modeval==1){
  if(box1manual==1 && b1manstatus==0){
    Serial.println("@@ BOX 1 MANUAL OPEN @@");
    box1open();
    b1manstatus=1;
  }
  else  if(box1manual==0 && b1manstatus==1){
    Serial.println("@@ BOX 1 MANUAL CLOSE @@");
    box1close();
    b1manstatus=0;
  }
}
void robopins_setup(){
  pinMode(MA1, OUTPUT);
  pinMode(MB1, OUTPUT);
}
void testrobo(){
  forwardrobo();
  delay(2000);
   stoprobo();
  delay(2000);
  backwardrobo() ;
   delay(2000);
```

```
  stoprobo();
    delay(2000);
  rightrobo();
    delay(2000);
  leftrobo();
    delay(2000);
    stoprobo();
}
void voicepinsetup(){
     pinMode(b1s1voicepin,OUTPUT);
     pinMode(b1s2voicepin,OUTPUT);
   digitalWrite(b1s1voicepin, 1);
    digitalWrite(b1s2voicepin, 1);
}
void b1s1voice(){
  Serial.println("^^^ B1S1 VOICE PLAY ^^^");
    digitalWrite(b1s1voicepin, 0);
    delay(300);
    digitalWrite(b1s1voicepin, 1);
}
void b1s2voice(){
  Serial.println("^^^ B1S2 VOICE PLAY ^^^");
    digitalWrite(b1s2voicepin, 0);
    delay(300);
    digitalWrite(b1s2voicepin, 1);
}
void testvoices(){
  b1s1voice();
  delay(2000);
  b1s2voice();
  delay(2000);
}
```

CHAPTER 9
RESULTS

The MediGuard Bot prototype was successfully developed on a mobile wheeled platform, integrating the ESP32 controller with essential components such as sensors, LCD, camera, and speaker module.

Fig. 9.1: ESP-32 powered Assistance Robot with Surveillance for Medicine Dispensing and Threat Detection

The image shows the LCD display of the MediGuard Bot system during the initialization phase, confirming that the bot is powered and operational. The message "IoT MEDIGUARD BOT SYSTEM" indicates successful system boot-up and readiness for operation.

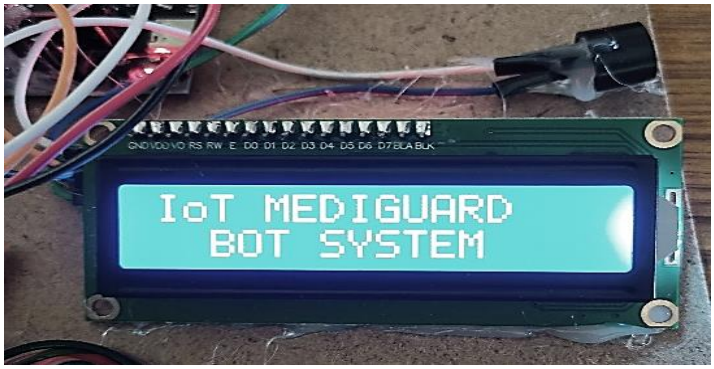

Fig. 9.2: System Initialization

The hardware setup of the MediGuard Bot, including the ESP32 microcontroller, LCD module, APAR voice module, speaker, flame sensor, gas sensor, buzzer and other essential electronic components.

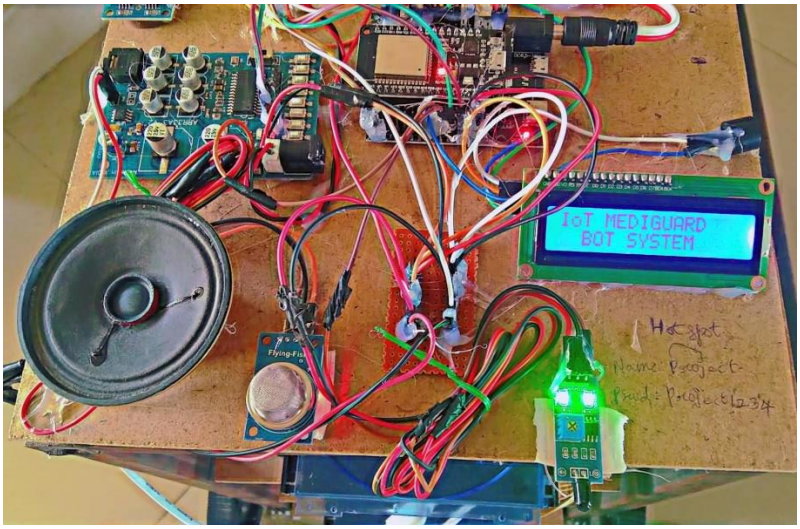

Fig. 9.3: Components of MediGuard Bot

The below images demonstrate the medicine dispensing mechanism of the MediGuard Bot. DVD loaders being activated to open the prototype of medicine box at scheduled intervals and APAR voice module with speaker deliver an audio alert to notify the patient that the medicine compartment has been opened.

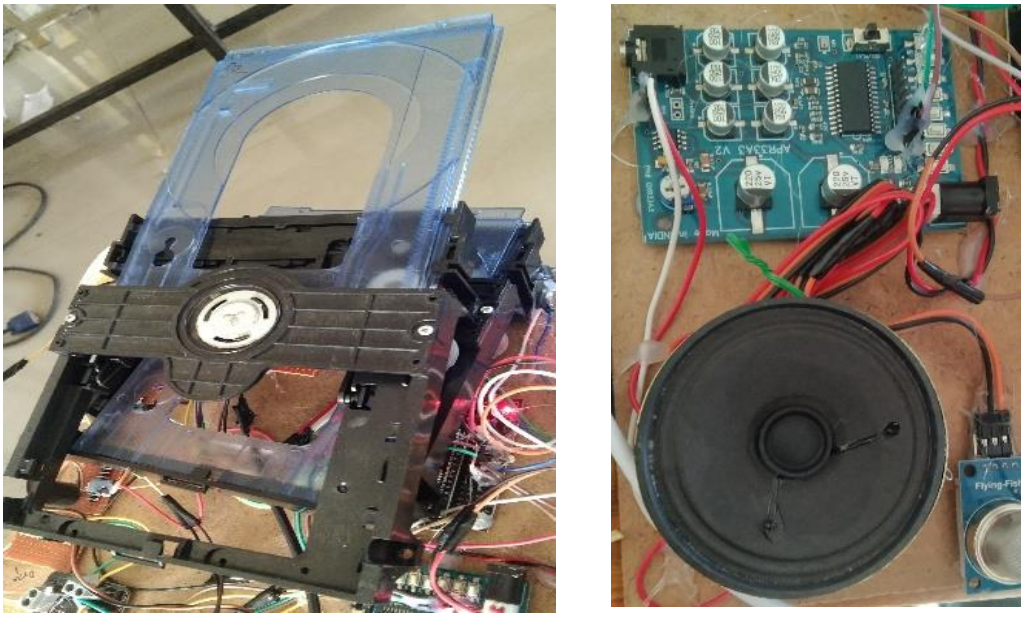

Fig. 9.4: Medicine Dispensing Functionality Modules (DVD Loader, Speaker)

The LCD screen of the MediGuard Bot serves as a real-time status console, displaying critical environmental conditions and system updates.

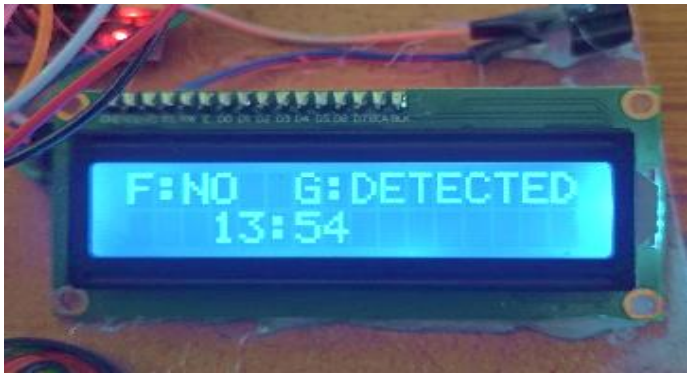

Fig. 9.5: LCD Console when Gas Detected

Fig. 9.6: LCD Console when Flame Identified

As shown in the images, it alerts “G: DETECTED” when hazardous gas is sensed in the surroundings, if not it gives “G: NORMAL”. Similarly, it indicates “F: YES” when a flame is identified, ensuring quick recognition of fire-related threats, if not it shows “F: NO” and also logs the event with a timestamp along with them.

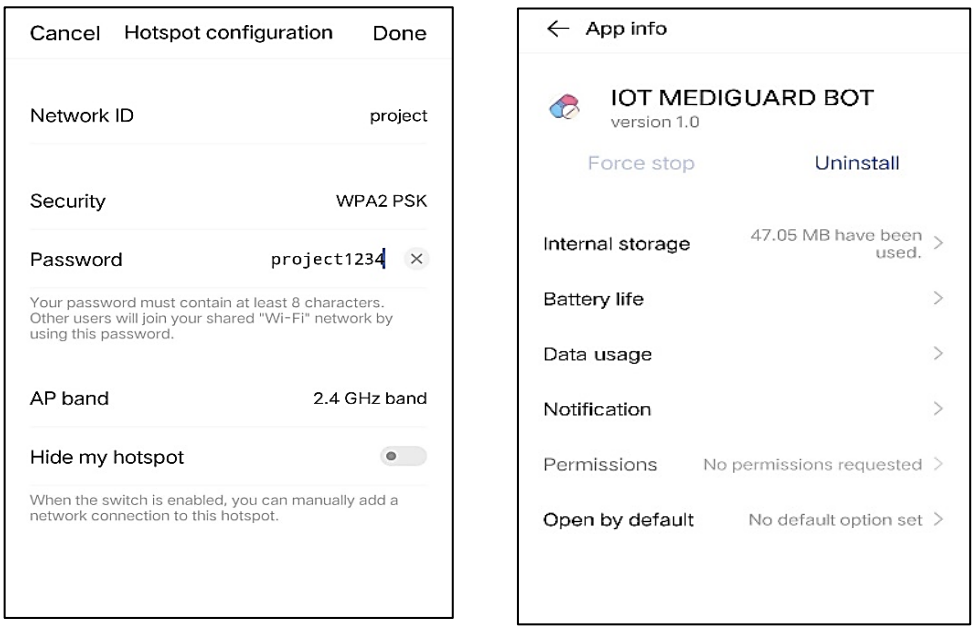

Fig. 9.7: Connecting MediGuard Bot with Mobile via App

The mobile interface of the "IoT MediGuard Bot" application. These confirms the successful installation and version details of the app and showcases the control dashboard, enabling manual operations such as opening and closing the medicine drives, monitoring gas and fire status, and navigating the robot. This user-friendly app enhances remote accessibility and real-time control of the MediGuard Bot.

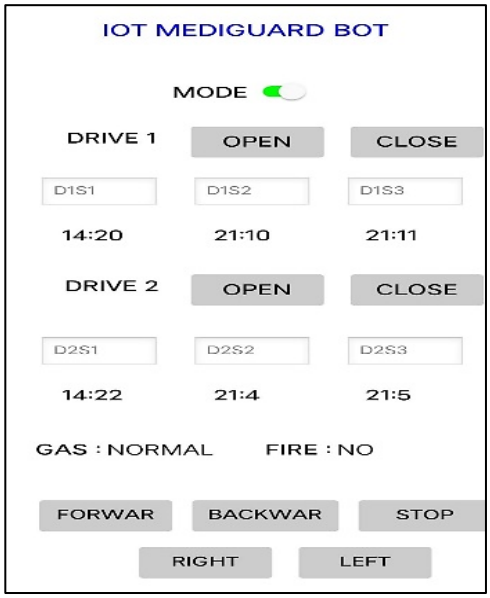

Fig. 9.8: App on Mobile Interface

Fig. 9.9: MediGuard Bot in Working

CHAPTER 10
CONCLUSION

The MediGuard Bot presents a comprehensive and reliable solution for modern healthcare and smart home security. By integrating multiple features such as scheduled medicine dispensing, real-time environmental monitoring, and remote surveillance, the project addresses key challenges faced by elderly and chronically ill individuals. The precise control offered by the RTC-based dispensing system ensures timely medication delivery, while the gas and flame sensors enhance safety by detecting hazardous conditions and initiating instant alerts. The inclusion of an IoT-based web interface and IP camera allows caregivers and family members to remotely monitor and manage the system, significantly improving responsiveness and peace of mind.

Overall, the project successfully demonstrates how microcontroller-based automation, particularly with the ESP32, can be harnessed to build intelligent, multifunctional systems that cater to real-world needs in healthcare and security. Unlike conventional robots, the MediGuard Bot offers manually guided mobility, enhancing its reliability and reducing the risk of errors in sensitive environments. Its modular design makes it adaptable for various applications, from hospitals to elderly care facilities and smart homes. By combining medicine dispensing, environmental threat detection, and surveillance into a single, user-friendly system, the MediGuard Bot stands as an innovative and practical contribution to the field of assistive robotics and healthcare technology.

CHAPTER 11

REFERENCES

[1] K. Ramesh, K. Rukmini, B. Abhilasha, V. Bhuvaneshwari, G. Pavan, 2024, Medicine Delivering and Patient Parameters Monitoring Robot, *Journal of Engineering Sciences*, Vol. 15.

[2] Aditi Dubey, Gandhamani CM, Meghashree M, Dr. Rekha N, 2023, Surveillance Robot Using ESP32 CAM Module, *International Advanced Research Journal in Science, Engineering and Technology*, Vol. 10.

[3] Prof. Y. B. Bachkar, A. Tekale, N. Pawar, 2024, IoT-Based Automatic Medicine Dispenser Using ESP32, *International Research Journal of Modernization in Engineering, Technology and Science*, Vol. 6.

[4] Asharuf, A., Krishnan, A., U. Karthik, Riya Mathew, & Prof. J. M. Thomas, 2023, Advanced Hospital Robot, *International Journal of Novel Research and Development*.

[5] Ismail, N., Anafi, F. O., & Sabur, A. A., 2023, Development of an Automatic Pill Dispensing and Health Vitals Measuring System for Hypertensive Patients, *World Journal of Sensors Network Research*.

[6] Pathiraja, M. A., and W. A. S. Wijesinghe. "IoT-Based Smart Medicine Dispenser." *2024 International Conference on Image Processing and Robotics (ICIPRoB)*. IEEE, 2024.

[7] Jinto, T. J., and Sudip Chakraborty. "ALEXA ENABLED MEDICINE REMINDER FOR ELDERLY PEOPLE USING AWS IOT, ESP MODULE."

[8] Kurniawan, Fery Andika, et al. "Implementation of an ESP-32 Wi-Fi CAM & Arduino-Based Robot for IoT Surveillance." *Seminar Nasional Teknik Elektro*. 2023.

[9] Geethanjali, P., and V. Ajay. "Ai-enhanced personal care robot assistant for hospital medication delivery." *2024 5th International Conference for Emerging Technology (INCET)*. IEEE, 2024.

[10] Kumari, M. Viajaya, et al. "Smart Drug Administration System using IoT-driven RTC Timer for Medication Management in Chronic Patients."

[11] Dayananda, P., and Amrutha G. Upadhya. "Development of Smart Pill Expert System Based on IoT." *Journal of The Institution of Engineers (India): Series B* 105.3 (2024): 457-467.

[12] Xu, Siyu, and Yizhuo Wen. "Design and Implementation of a Remote Monitoring Robot System for Solitary Elderly Individuals." *2024 39th Youth Academic Annual Conference of Chinese Association of Automation (YAC)*. IEEE, 2024.

[13] Arif, Md Ahsan, et al. "Improved the accuracy of IOT based LPG leakage detection system with early fire prediction and smart alert." *International Journal of Scientific Research and Management (IJSRM)* 12.04 (2024): 1110-1116.

[14] Upadhyaya, Ajay N., et al. "Advancing Safety: IoT-Based Multi Sensor System for Real-Time Multiple Hazards Detection and Alarming." *2023 4th International Conference on Smart Electronics and Communication (ICOSEC)*. IEEE, 2023.

[15] Cruces, Alejandro, et al. "Socially Assistive Robots in Smart Environments to Attend Elderly People—A Survey." *Applied Sciences* 14.12 (2024): 5287.

[16] Vimal, S., et al. "A Smart Pill Dispenser with Real Time Alerts, Remote Access, and Enhanced Safety." *2024 International Conference on Computing and Intelligent Reality Technologies (ICCIRT)*. IEEE, 2024.

[17] Meem, Md Mominur Rahman, et al. "Artificially Intelligent Surveillance and Security Sentinel for Technologically Enhanced and Protected Communities."

[18] Suzilawati, M., Muhammad Nur Farhan Nasir Ali, and Mohd Hazri Mohd Rusli. "IoT Robotic Medication Dispenser for Elderly Care." *International Conference on Innovation & Entrepreneurship in Computing, Engineering & Science Education (InvENT 2024)*. Atlantis Press, 2024.

[19] Senthil, G. A., et al. "An enhanced smart intelligent detecting and alerting system for industrial gas leakage using IoT in sensor network." *2023 5th International Conference on Smart Systems and Inventive Technology (ICSSIT)*. IEEE, 2023.

[20] Ilhamdi, Luthvy, Muhammad Fikry, and Hafizh Al Kautsar Aidilof. "Smart Fire Prevention: An IoT Approaceh to Detecting LPG Leaks and Fire

Hazards." *Proceedings of International Conference on Multidisciplinary Engineering (ICOMDEN)*. Vol. 2. 2024.

[21] Darwante, N. K., et al. "Hybrid model for robotic surveillance using advance computing techniques with IoT." *2023 3rd International Conference on Pervasive Computing and Social Networking (ICPCSN)*. IEEE, 2023.

[22] Haido, Mohammad Garba, Abdulrahman Ebrahim Mansoor Kamel, and Ziya'ulhaq Haruna. *Mobile Robot for Remote Surveillance through Video Monitoring*. Diss. Department of Electrical and Elecrtonics Engineering (EEE), Islamic University of Technology (IUT), Board Bazar, Gazipur, Bangladesh, 2024.

[23] Paul, Liton Chandra, et al. "A smart medicine reminder kit with mobile phone calls and some health monitoring features for senior citizens." *Heliyon* 10.4 (2024).

[24] Vardakis, George, et al. "Review of Smart-Home Security Using the Internet of Things." *Electronics* 13.16 (2024): 3343.

[25] Kanagamalliga, S., and S. Rajalingam. "Innovative Fire and Gas Recognition System Featuring Remote Monitoring and Automated Alerts." *Procedia Computer Science* 252 (2025): 7-14.

[26] Kadiriboiena, Venkata Sai, et al. "IoT-Powered Smart Surveillance Sentinel Bot." *2024 IEEE Students Conference on Engineering and Systems (SCES)*. IEEE, 2024.

[27] Roy, Souptik, et al. "An Integrated AI Doctor Assistant Bot with Facial Emotion Recognition and Smart Medication Dispensing for Enhanced Patient Care." *AJEC* (2025).

[28] Oladele, Oluwaseyi Kolawole. "AI in Aging and Elder Care: Assistive Technologies, Health Monitoring, and Social Robotics." (2025).

[29] Sudarmani, R., J. Harshitha, and K. Rajakumari. "Development of Smart Automatic Drug Dispenser for Elderly and Disabled People." *2024 International Conference on Science Technology Engineering and Management (ICSTEM)*. IEEE, 2024.

[30] Elavarasan, Arish Krishna, et al. "Healthcare System for Scheduled Medicine Dispensing and Reminder for Elderly People." *International Journal* 8.3 (2023).